

**BỘ GIÁO DỤC VÀ ĐÀO TẠO**

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HCM**

**KHOA: ĐIỆN – ĐIỆN TỬ**

**MÔN: ĐỒ ÁN 1**

-----oOo-----



**HCM-UTE**

**BÁO CÁO ĐỒ ÁN 1**

**HỆ THỐNG KHÓA CỬA THÔNG MINH**

**GVHD: PGS.TS Phạm Ngọc Sơn**

**Nhóm : 2**

**SVTH: Lý Hoàng Khang      23161133**

**Lý Vĩnh Khang      23161134**

**KHÓA : 2023-2027**

**NGÀNH : CNKT ĐIỆN TỬ - VIỄN THÔNG**

**Tp. Hồ Chí Minh, 8 tháng 7 năm 2026**



## **LỜI CẢM ƠN**

Trong suốt quá trình thực hiện đồ án 1, em đã nhận được rất nhiều sự hỗ trợ, hướng dẫn và động viên từ quý thầy cô, gia đình và bạn bè. Em xin gửi lời cảm ơn chân thành đến

Trước tiên, em xin bày tỏ lòng biết ơn sâu sắc đến Thầy Phạm Ngọc Sơn – giảng viên hướng dẫn đã tận tình chỉ dẫn, góp ý và định hướng cho em trong từng bước thực hiện đề tài. Những kiến thức và kinh nghiệm quý báu mà Thầy/Cô chia sẻ là nền tảng giúp em hoàn thành báo cáo này.

Em cũng xin cảm ơn quý Thầy Cô trong Khoa Điện – Điện tử đã truyền đạt kiến thức chuyên môn trong suốt ba năm học vừa qua, tạo điều kiện và cơ sở vật chất để em có thể triển khai mô hình thực tế. Cuối cùng, em xin cảm ơn gia đình và các bạn cùng nhóm đã luôn hỗ trợ, chia sẻ và đồng hành trong quá trình thực hiện đồ án.

Do còn nhiều hạn chế về kiến thức và kinh nghiệm, báo cáo không tránh khỏi những thiếu sót. Em rất mong nhận được sự nhận xét, góp ý từ quý Thầy để hoàn thiện hơn trong các giai đoạn tiếp theo.

## TÓM TẮT ĐỀ TÀI

Đề tài "Thiết kế và thi công hệ thống khóa cửa thông minh sử dụng ESP32, RFID và Firebase" được thực hiện nhằm xây dựng một giải pháp bảo mật cửa ra vào có khả năng điều khiển đa phương thức, phù hợp với xu hướng nhà thông minh (Smart Home) hiện nay.

Hệ thống cho phép người dùng mở khóa thông qua ba phương thức chính: nhập mật khẩu trên màn hình cảm ứng TFT SPI, quét thẻ RFID RC522 điều khiển từ xa qua ứng dụng điện thoại kết nối Firebase Realtime Database và nhận diện khuôn mặt. Giao diện người dùng được thiết kế trực quan trên màn hình ILI9341 240x320, sử dụng keypad ảo cảm ứng thay thế cho phím bấm vật lý và có nguồn dự phòng khi cúp điện.

Hệ thống được lập trình trên vi điều khiển ESP32 với khả năng xử lý đa nhiệm đảm bảo các tác vụ như đọc thẻ RFID, kiểm tra Firebase và điều khiển giao diện hoạt động song song mà không gây xung đột. Dữ liệu mật khẩu và danh sách thẻ RFID được lưu trữ bền vững trong bộ nhớ Flash (NVS) của ESP32.

Từ khóa: ESP32, RFID, Firebase, nhận diện khuôn mặt, TFT cảm ứng, khóa cửa thông minh, IoT.

## MỤC LỤC

|                                                   |    |
|---------------------------------------------------|----|
| LỜI CẢM ƠN .....                                  | 3  |
| TÓM TẮT ĐỀ TÀI .....                              | 4  |
| MỤC LỤC .....                                     | 5  |
| DANH MỤC HÌNH ẢNH .....                           | 7  |
| PHẦN 1 : QUÁ TRÌNH THIẾT KẾ .....                 | 10 |
| CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI .....                  | 10 |
| 1.1. Đặt vấn đề .....                             | 10 |
| 1.2. Mục tiêu đề tài .....                        | 10 |
| 1.3. Giới hạn đề tài.....                         | 11 |
| CHƯƠNG 2: CƠ SỞ LÝ THUYẾT .....                   | 12 |
| 2.1. Vi điều khiển ESP32 .....                    | 12 |
| 2.1.1. Giới thiệu chung.....                      | 12 |
| 2.1.2. Thông số kỹ thuật chính.....               | 12 |
| 2.1.3. FreeRTOS và đa nhiệm trên ESP32.....       | 12 |
| 2.2. Màn hình TFT cảm ứng SPI (ILI9341).....      | 13 |
| 2.2.1. Giới thiệu màn hình ILI9341 .....          | 13 |
| 2.2.2. Cảm ứng điện trở XPT2046.....              | 13 |
| 2.3. Module RFID RC522.....                       | 14 |
| 2.3.1. Nguyên lý hoạt động RFID.....              | 14 |
| 2.3.2. Thông số kỹ thuật RC522 .....              | 14 |
| 2.4. Module Relay và khóa điện từ .....           | 14 |
| 2.4.1. Nguyên lý hoạt động Relay.....             | 15 |
| 2.4.2. Khóa điện từ (Solenoid Lock).....          | 15 |
| 2.5. Firebase Realtime Database .....             | 16 |
| 2.5.1. Giới thiệu Firebase.....                   | 16 |
| 2.5.2. Cách ESP32 giao tiếp với Firebase.....     | 17 |
| 2.6. Ứng dụng MIT App Inventor.....               | 18 |
| 2.6.1. Giới thiệu MIT App Inventor.....           | 18 |
| 2.6.2. Tích hợp Firebase trong App Inventor.....  | 19 |
| 2.6.3. Thiết kế Blocks điều khiển .....           | 19 |
| 2.7. Các Linh kiện phụ khác .....                 | 20 |
| 2.7.1. Adapter 220V - 12V DC (Nguồn sơ cấp) ..... | 20 |
| 2.7.2. Mạch giảm áp LM2596S 3A.....               | 20 |

|                                                         |    |
|---------------------------------------------------------|----|
| 2.7.3. XY-850 Mạch chuyển nguồn sạc dự phòng .....      | 21 |
| 2.7.4. Cảm biến từ MC-38 .....                          | 22 |
| 2.7.5. Cell pin 12V (Nguồn dự phòng) .....              | 23 |
| CHƯƠNG 3: THIẾT KẾ HỆ THỐNG .....                       | 24 |
| 3.1. Sơ đồ khối tổng thể .....                          | 24 |
| 3.2. Sơ đồ mạch nguyên lý và kết nối dây .....          | 24 |
| 3.3. Lưu đồ thuật toán .....                            | 25 |
| 3.3.1. Khởi động .....                                  | 25 |
| 3.3.2. Vòng lặp xử lý các tác vụ .....                  | 28 |
| 3.3.3. Nhập mật khẩu .....                              | 30 |
| 3.3.4. Quét thẻ Rfid.....                               | 33 |
| 3.3.5. Trang APP/FIREBASE (PAGE_APP) .....              | 35 |
| 3.3.6. Luồng đổi mật khẩu .....                         | 39 |
| 3.3.7. Luồng thêm và xóa thẻ Rfid.....                  | 41 |
| 3.3.8. Luồng xử lý relay và MC-38 .....                 | 44 |
| 3.3.9. Các luồng hoạt động các task nền song song.....  | 48 |
| 3.4. Giao diện màn hình cảm ứng .....                   | 50 |
| 3.5. Giao diện ứng dụng App Inventor .....              | 53 |
| 3.6. Thực hiện Firebase.....                            | 54 |
| 3.7. Mô hình cửa thiết kế .....                         | 55 |
| PHẦN 2: KẾT QUẢ .....                                   | 57 |
| CHƯƠNG 4: KẾT QUẢ VÀ ĐÁNH GIÁ .....                     | 57 |
| 4.1. Thiết kế và thi công mạch PCB: .....               | 57 |
| 4.1.1. Thiết kế Schematic các khối của mạch PCB .....   | 57 |
| 4.1.2. Thiết kế mạch in PCB.....                        | 57 |
| 4.2. Thi công .....                                     | 58 |
| 4.3. Mô hình cánh cửa thực tế sau khi hoàn thành: ..... | 59 |
| 4.4. Kết quả kiểm tra các chức năng .....               | 61 |
| 4.5. Nhận xét và Hạn chế.....                           | 62 |
| CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....             | 64 |
| 5.1. Kết luận.....                                      | 64 |
| 5.2. Hướng phát triển .....                             | 65 |
| TÀI LIỆU KHAM KHẢO .....                                | 68 |

## DANH MỤC HÌNH ẢNH

|                                                                    |    |
|--------------------------------------------------------------------|----|
| Hình 1: ESP32 DevKit V1 (38 pins) và sơ đồ chân.....               | 12 |
| Hình 2 : Màn hình cảm ứng TFT 2.8 inch SPI ILI9341 .....           | 13 |
| Hình 3 : RFID MFRC-522 NFC 13.56MHz.....                           | 14 |
| Hình 4 : Module 2 relay 5V với opto cách ly kích H/L .....         | 15 |
| Hình 5 : LY-01 Khóa Điện Solenoid 12V .....                        | 15 |
| Hình 6: Mô hình cơ bản của Firebase kết nối với Esp32.....         | 16 |
| Hình 7 : MIT App Inventor.....                                     | 18 |
| Hình 8 : Mạch giảm áp LM2596 4A.....                               | 20 |
| Hình 9 : XY-850 Mạch chuyển nguồn sạc dự phòng .....               | 21 |
| Hình 10 : Cảm biến từ MC-38 .....                                  | 22 |
| Hình 11 : Cell pin 12V.....                                        | 23 |
| Hình 12 : Sơ đồ khối tổng thể.....                                 | 24 |
| Hình 13 : Sơ đồ mạch nguyên lý.....                                | 24 |
| Hình 14 : Lưu đồ khởi động.....                                    | 25 |
| Hình 15 : Lưu đồ thuật toán vòng lặp xử lý.....                    | 28 |
| Hình 16 : Lưu đồ thuật toán phương thức nhập MẬT KHẨU .....        | 31 |
| Hình 17 : Lưu đồ thuật toán phương thức thẻ Rfid.....              | 33 |
| Hình 18 : Lưu đồ thuật toán phương thức APP .....                  | 36 |
| Hình 19 : Lưu đồ thuật toán đồ mật khẩu .....                      | 39 |
| Hình 20 : Lưu đồ thuật toán thêm hoặc xóa thẻ Rfid.....            | 41 |
| Hình 21 : Lưu đồ thuật toán xử lý relay và cảm biến từ MC-38 ..... | 45 |
| Hình 22 : Luồng hoạt động các task nền song song .....             | 49 |
| Hình 23 : Giao diện trang chủ.....                                 | 50 |
| Hình 24 : Giao diện trang nhập mật khẩu .....                      | 50 |
| Hình 25 : Giao diện trang mở cửa bằng APP .....                    | 51 |
| Hình 26 : Giao diện trang tùy chọn.....                            | 51 |
| Hình 27 : Giao diện trang xác nhận đổi mật khẩu .....              | 51 |
| Hình 28 : Giao diện trang nhập mật khẩu mới.....                   | 52 |
| Hình 29 : Giao diện trang xác nhận thêm/xóa thẻ Rfid .....         | 52 |
| Hình 30 : Giao diện trang quản lý thẻ Rfid.....                    | 53 |

|                                                            |    |
|------------------------------------------------------------|----|
| Hình 31 : Giao diện thêm thẻ Rfid.....                     | 53 |
| Hình 32 : Giao diện ứng dụng App Inventor .....            | 54 |
| Hình 33 : Trang Firebase Realtime Database .....           | 55 |
| Hình 34 : Ý tưởng mô hình cửa .....                        | 56 |
| Hình 35 : Mô hình cửa đã được thiết kế .....               | 56 |
| Hình 36 : Schematic các khối .....                         | 57 |
| Hình 37 : PCB sau khi vẽ xong.....                         | 57 |
| Hình 38 : Mạch PCB sau khi in .....                        | 58 |
| Hình 39 : Khối cung cấp nguồn .....                        | 58 |
| Hình 40 : Mạch trung tâm sau khi hàn và lắp linh kiện..... | 59 |
| Hình 41 : Phía trước mô hình cánh cửa .....                | 59 |
| Hình 42 : Phía sau mô hình cánh cửa .....                  | 60 |
| Hình 43 : Nơi đặt mạch điều khiển và khối nguồn .....      | 60 |



## **DANH SÁCH CÁC BẢNG**

|                                                               |    |
|---------------------------------------------------------------|----|
| Bảng 1 : Tổng hợp các phương thức REST API của Firebase ..... | 17 |
| Bảng 2 : Đặc tính kỹ thuật của module LM2596.....             | 21 |
| Bảng 3 : Thông số kỹ thuật của cell pin.....                  | 23 |
| Bảng 4 : Kết nối dây của mô hình.....                         | 25 |
| Bảng 5 : Kết quả kiểm thử chức năng của hệ thống .....        | 61 |

## **PHẦN 1 : QUÁ TRÌNH THIẾT KẾ**

### **CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI**

#### **1.1. Đặt vấn đề**

Trong bối cảnh cuộc sống hiện đại ngày càng phát triển, nhu cầu bảo mật nhà ở và các công trình dân dụng trở nên cấp thiết hơn bao giờ hết. Các hệ thống khóa cửa truyền thống sử dụng chìa khóa cơ học tồn tại nhiều hạn chế như nguy cơ mất chìa, làm chìa giả, không thể kiểm soát từ xa và thiếu nhật ký truy cập.

Sự bùng nổ của công nghệ Internet of Things (IoT) đã mở ra hướng giải quyết hiệu quả cho bài toán trên. Các hệ thống khóa cửa thông minh ngày nay cho phép xác thực đa lớp, điều khiển từ xa qua điện thoại thông minh và ghi nhận lịch sử ra vào – nâng cao đáng kể mức độ an toàn và tiện lợi cho người dùng.

Xuất phát từ thực tiễn đó, nhóm đề xuất thực hiện đề tài "Thiết kế và thi công hệ thống khóa cửa thông minh sử dụng ESP32, RFID và Firebase" nhằm xây dựng một mô hình hoàn chỉnh, có tính ứng dụng thực tế cao và phù hợp với xu hướng nhà thông minh hiện nay.

#### **1.2. Mục tiêu đề tài**

Đề tài được thực hiện với các mục tiêu cụ thể sau:

Thiết kế và thi công hệ thống khóa cửa thông minh hoạt động ổn định, tin cậy.

Xây dựng giao diện người dùng trực quan trên màn hình TFT cảm ứng SPI với keypad ảo.

Tích hợp xác thực đa phương thức: mật khẩu số, thẻ RFID và điều khiển qua ứng dụng Firebase.

Bảo mật hệ thống với mã xác nhận Admin, cơ chế khóa tạm khi nhập sai nhiều lần.

Lưu trữ dữ liệu bền vững vào bộ nhớ Flash của ESP32 (NVS/Preferences).

Lập trình ứng dụng điện thoại bằng MIT App Inventor kết nối Firebase để điều khiển từ xa.

### 1.3. Giới hạn đề tài

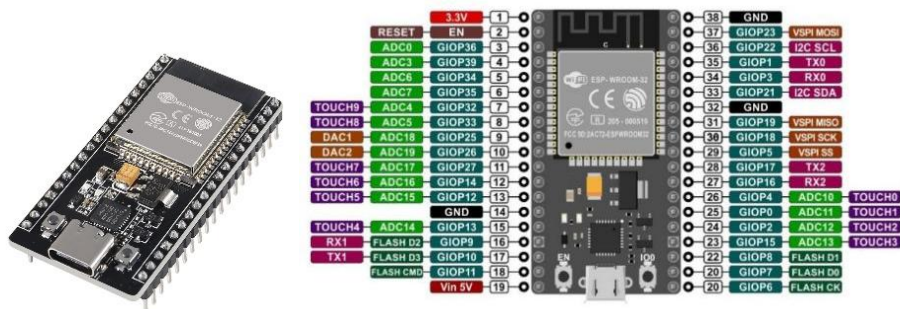
Trong phạm vi Đề án 1, đề tài tập trung vào các nội dung sau và có một số giới hạn nhất định:

- Hệ thống được thi công ở dạng mô hình trình diễn (prototype), chưa lắp đặt thực tế trên công trình.
- Ứng dụng điều khiển được xây dựng trên nền tảng MIT App Inventor, chỉ hỗ trợ hệ điều hành Android.
- Kết nối Firebase yêu cầu ESP32 có kết nối WiFi ổn định; chưa xử lý trường hợp mất kết nối lâu dài.
- Mã nguồn và thiết kế tập trung vào tính năng cốt lõi; chưa tối ưu hóa tiêu thụ điện năng.
- Bảo mật ở mức cơ bản với xác thực thẻ RFID; chưa tích hợp các lớp bảo mật nâng cao như mã hóa dữ liệu đầu cuối hay xác thực hai yếu tố.

## CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

### 2.1. Vi điều khiển ESP32

#### 2.1.1. Giới thiệu chung



Hình 1: ESP32 DevKit V1 (38 pins) và sơ đồ chân

ESP32 là vi điều khiển do công ty Espressif Systems (Trung Quốc) phát triển, ra mắt năm 2016. ESP32 tích hợp hai nhân xử lý Xtensa LX6 32-bit với tốc độ lên đến 240 MHz, cùng với bộ nhớ SRAM 520 KB và khả năng kết nối WiFi 802.11 b/g/n và Bluetooth 4.2/BLE trong cùng một chip.

So với thế hệ ESP8266 trước đó, ESP32 vượt trội hơn về hiệu năng xử lý, số lượng chân GPIO (38 chân), hỗ trợ giao thức SPI, I2C, UART, I2S, ADC, DAC và đặc biệt là khả năng chạy hệ điều hành thời gian thực FreeRTOS tích hợp sẵn.

#### 2.1.2. Thông số kỹ thuật chính

Lõi xử lý: Dual-core Xtensa LX6, tốc độ tối đa 240 MHz

Bộ nhớ: SRAM 520 KB, Flash ngoài từ 4 MB đến 16 MB

WiFi: 802.11 b/g/n (2.4 GHz), Bluetooth 4.2 và BLE

GPIO: 34 chân I/O lập trình được, hỗ trợ PWM, ADC (18 kênh), DAC (2 kênh)

Giao tiếp: 4x SPI, 2x I2C, 2x I2S, 3x UART

Điện áp hoạt động: 3.3 V, điện áp cấp nguồn: 5 V qua cổng USB

Hỗ trợ FreeRTOS đa nhiệm đa lõi

#### 2.1.3. FreeRTOS và đa nhiệm trên ESP32

FreeRTOS (Free Real-Time Operating System) là hệ điều hành thời gian thực mã nguồn mở được tích hợp sẵn trong framework Arduino và ESP-IDF của ESP32. FreeRTOS

cho phép lập trình viên tạo nhiều task chạy song song, phân chia trên hai nhân CPU của ESP32.

Trong đề tài này, FreeRTOS được ứng dụng để phân tách các tác vụ: task đọc Firebase và task quét RFID chạy trên Core 0, trong khi xử lý giao diện cảm ứng và relay chạy trên Core 1. Cơ chế Mutex (xSemaphoreCreateMutex) được sử dụng để bảo vệ tài nguyên dùng chung là relay, tránh xung đột khi nhiều task cùng cố gắng điều khiển.

## **2.2. Màn hình TFT cảm ứng SPI (ILI9341)**

### **2.2.1. Giới thiệu màn hình ILI9341**



*Hình 2 : Màn hình cảm ứng TFT 2.8 inch SPI ILI9341*

ILI9341 là driver IC phổ biến cho màn hình TFT LCD màu 240x320 pixel (2.4 inch hoặc 2.8 inch). Màn hình hỗ trợ giao tiếp SPI tốc độ cao, có khả năng hiển thị 262.144 màu (18-bit) và được sử dụng rộng rãi trong các dự án nhúng nhờ mức giá thấp và thư viện hỗ trợ phong phú.

Trong đề tài này, thư viện TFT\_SPI của Bodmer được sử dụng để điều khiển màn hình, cho phép vẽ hình học, text, bitmap và hỗ trợ font tùy chỉnh với hiệu năng cao thông qua DMA SPI.

### **2.2.2. Cảm ứng điện trở XPT2046**

XPT2046 là chip điều khiển màn hình cảm ứng điện trở 4 dây, thường được tích hợp cùng màn hình ILI9341. Chip giao tiếp với ESP32 qua giao thức SPI, cung cấp tọa độ chạm (x, y) và áp lực chạm (z) dưới dạng giá trị ADC 12-bit.

Để chuyển đổi tọa độ raw từ XPT2046 sang tọa độ pixel thực tế trên màn hình, cần thực hiện hiệu chỉnh (calibration) bằng hàm map() với bốn tham số TS\_MINX, TS\_MAXX, TS\_MINY, TS\_MAXY đo thực tế từng màn hình.

## 2.3. Module RFID RC522

### 2.3.1. Nguyên lý hoạt động RFID



*Hình 3 : RFID MFRC-522 NFC 13.56MHz*

RFID (Radio Frequency Identification) là công nghệ nhận dạng đối tượng sử dụng sóng radio. Hệ thống RFID gồm hai thành phần chính: đầu đọc (Reader) và thẻ (Tag/Card). Đầu đọc phát ra sóng điện từ, cung cấp năng lượng cho thẻ thụ động và kích hoạt quá trình truyền dữ liệu UID (Unique Identifier).

Module RC522 hoạt động ở tần số 13.56 MHz, tương thích với chuẩn ISO 14443A/MIFARE. Mỗi thẻ/tag RFID mang một mã UID duy nhất từ 4 đến 10 byte, không thể thay đổi, được dùng để định danh và xác thực trong hệ thống.

### 2.3.2. Thông số kỹ thuật RC522

Tần số hoạt động: 13.56 MHz

Chuẩn hỗ trợ: ISO 14443A, MIFARE Classic/Ultralight/Plus

Giao tiếp: SPI (tốc độ tối đa 10 Mbps), I2C, UART

Điện áp hoạt động: 2.5 V – 3.3 V

Khoảng cách đọc thẻ: 0 – 5 cm tùy loại thẻ và môi trường

Thư viện Arduino: MFRC522 (GithubCommunity)

## 2.4. Module Relay và khóa điện từ

### 2.4.1. Nguyên lý hoạt động Relay



*Hình 4 : Module 2 relay 5V với opto cách ly kích H/L*

Relay là thiết bị đóng ngắt điện tử hoạt động dựa trên nguyên lý điện từ. Khi cuộn dây trong relay được cấp dòng điện nhỏ (từ GPIO ESP32), từ trường sinh ra sẽ hút thanh tiếp điểm, đóng hoặc mở mạch điện có điện áp/dòng điện lớn hơn ở phía tải.

Trong đề tài này, relay 5V 1 kênh được sử dụng để điều khiển khóa điện từ (solenoid lock) hoạt động ở 12V DC. ESP32 xuất tín hiệu HIGH/LOW qua GPIO để điều khiển relay, qua đó gián tiếp điều khiển cơ cấu khóa cửa.

### 2.4.2. Khóa điện từ (Solenoid Lock)



*Hình 5 : LY-01 Khóa Điện Solenoid 12V*

Khóa điện từ là loại khóa sử dụng nam châm điện để giữ hoặc nhả chốt khóa. Khi có điện, chốt khóa được thu vào (Normally Closed – NC) hoặc thả ra (Normally Open – NO) tùy theo thiết kế.

Khóa điện Solenoid LY-01 12V gồm các bộ phận chính sau:

Cuộn dây Solenoid là cuộn dây đồng quấn quanh lõi sắt. Khi cấp điện sẽ tạo ra từ trường để hút lõi sắt chuyển động.

Lõi sắt từ (plunger) là thanh sắt chuyển động tịnh tiến bên trong cuộn dây dưới tác dụng của lực từ. Bộ phận này liên kết trực tiếp với chốt khóa.

Chốt khóa thường là chốt tròn bằng thép không gỉ, dùng để giữ hoặc nhả cửa khi hoạt động.

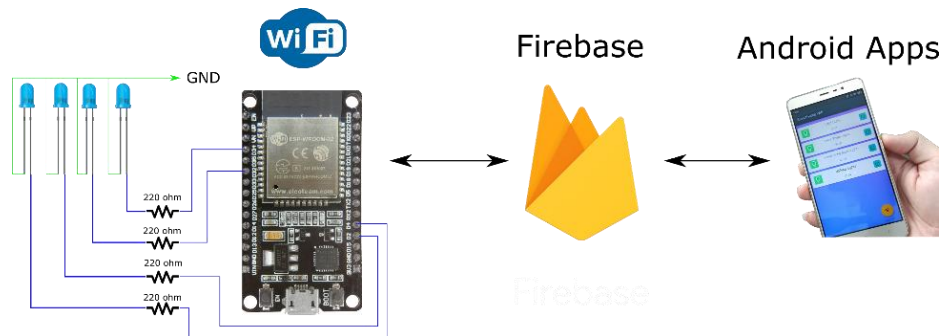
Lò xo hồi có nhiệm vụ đẩy chốt trở về vị trí ban đầu khi ngắt điện.

Vỏ khóa kim loại được làm bằng thép hoặc hợp kim kim loại giúp bảo vệ các linh kiện bên trong và tăng độ bền cơ học.

Dây cấp nguồn dùng để cấp nguồn DC 12V cho cuộn Solenoid hoạt động

## 2.5. Firebase Realtime Database

### 2.5.1. Giới thiệu Firebase



Hình 6: Mô hình cơ bản của Firebase kết nối với Esp32

Firebase là nền tảng phát triển ứng dụng của Google, cung cấp nhiều dịch vụ đám mây như Authentication, Cloud Firestore, Realtime Database, Cloud Storage và Hosting. Firebase được Google mua lại vào năm 2014 và phát triển thành một nền tảng BaaS (Backend-as-a-Service) toàn diện. Thay vì tự xây dựng và quản lý server, lập trình viên có thể sử dụng trực tiếp các dịch vụ sẵn có của Firebase để tập trung vào logic ứng dụng. Các dịch vụ nổi bật gồm:

Authentication xác thực người dùng qua email/mật khẩu, Google, Facebook, GitHub hoặc ẩn danh

Cloud Firestore database NoSQL thế hệ mới, hỗ trợ query phức tạp và mở rộng tốt hơn

Realtime Database database NoSQL truyền thống, tối ưu cho đồng bộ dữ liệu tức thì

Cloud Storage lưu trữ file nhị phân như ảnh, video, tài liệu

Hosting triển khai web tĩnh với CDN toàn cầu

Firebase Realtime Database là cơ sở dữ liệu NoSQL lưu trữ dữ liệu dạng JSON, hỗ trợ đồng bộ dữ liệu theo thời gian thực giữa các thiết bị.



### 2.5.2. Cách ESP32 giao tiếp với Firebase

Khác với database quan hệ truyền thống (MySQL, PostgreSQL) lưu dữ liệu dạng bảng, Firebase Realtime Database lưu toàn bộ dữ liệu dưới dạng một cây JSON duy nhất trên đám mây. Điểm mạnh cốt lõi là cơ chế đồng bộ thời gian thực (real-time sync) khi một thiết bị thay đổi dữ liệu, tất cả các thiết bị đang kết nối và lắng nghe đều nhận được cập nhật ngay lập tức thông qua WebSocket, mà không cần client phải hỏi lại server theo chu kỳ.

Cơ chế này phù hợp với nhiều ứng dụng như chat, theo dõi vị trí, điều khiển thiết bị IoT từ xa, hay hiển thị dashboard cảm biến theo thời gian thực.

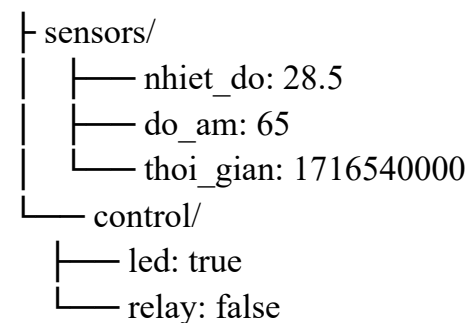
Dữ liệu trong Firebase Realtime Database được tổ chức dạng cây JSON và có thể truy xuất qua REST API bằng giao thức HTTP/HTTPS.

Toàn bộ database là một cây JSON không có bảng, không có schema cố định. Mỗi nút (node) trong cây đều có một URL riêng biệt, cho phép đọc hoặc ghi trực tiếp vào bất kỳ vị trí nào trong cây thông qua REST API.

Ví dụ: [https://ten-project-default-rtdb.firebaseio.com/sensors/nhiet\\_do.json](https://ten-project-default-rtdb.firebaseio.com/sensors/nhiet_do.json)

Cấu trúc cây điển hình trong một dự án IoT:

/ (root)



REST API hỗ trợ đầy đủ các thao tác CRUD thông qua HTTP method:

*Bảng 1 : Tổng hợp các phương thức REST API của Firebase*

| Method | Thao tác                                       |
|--------|------------------------------------------------|
| GET    | Đọc dữ liệu tại một node                       |
| PUT    | Ghi/ghi đè toàn bộ node                        |
| PATCH  | Cập nhật một số field, giữ nguyên phần còn lại |

|        |                                        |
|--------|----------------------------------------|
| POST   | Thêm node con mới với key tự động sinh |
| DELETE | Xóa node                               |

Điều này giúp ESP32 có thể đọc/ghi dữ liệu lên Firebase mà không cần SDK chuyên biệt, chỉ cần thư viện HTTPClient.

ESP32 là vi điều khiển có tích hợp WiFi, có khả năng gửi request HTTP/HTTPS như bất kỳ client web nào. Nhờ Firebase hỗ trợ REST API chuẩn, ESP32 chỉ cần thư viện HTTPClient vốn có sẵn trong Arduino framework để tương tác hoàn toàn với Firebase mà không cần cài thêm SDK riêng.

Luồng hoạt động cơ bản:

Cảm biến → ESP32 (đọc giá trị) → HTTPClient (đóng gói JSON) → HTTPS PUT/PATCH → Firebase Realtime DB

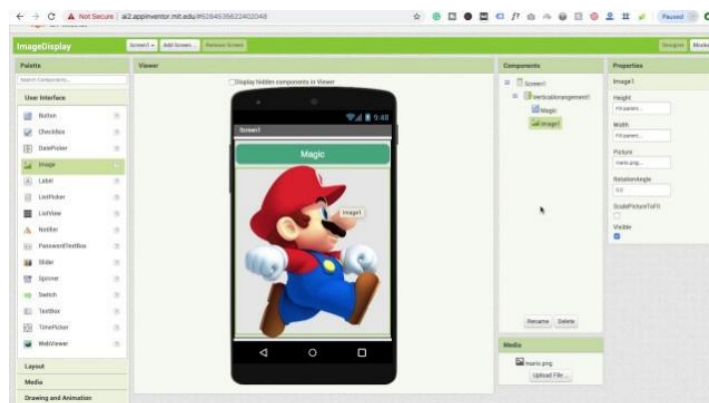
↑↓ đồng bộ

App mobile / Web dashboard

ESP32 cũng có thể đọc ngược lại gọi GET định kỳ để nhận lệnh điều khiển từ người dùng gửi qua app, sau đó tác động lên relay, LED hoặc motor tương ứng. Đây là mô hình IoT hai chiều đơn giản, không cần server trung gian, phù hợp cho các dự án nhà thông minh, giám sát môi trường và tự động hóa quy mô nhỏ.

## 2.6. Ứng dụng MIT App Inventor

### 2.6.1. Giới thiệu MIT App Inventor



Hình 7 : MIT App Inventor

MIT App Inventor là một công cụ phát triển ứng dụng Android trực tuyến, miễn phí do MIT (Massachusetts Institute of Technology) phát triển. Điểm đặc biệt là lập trình bằng giao diện kéo thả khối lệnh (block-based programming) - tương tự Scratch - thay vì viết code truyền thống, giúp người mới bắt đầu có thể tạo ra ứng dụng Android mà không cần biết Java hay Kotlin.

Đặc điểm nổi bật:

Chạy hoàn toàn trên trình duyệt web, không cần cài phần mềm

Giao diện chia làm 2 phần: Designer (thiết kế giao diện) và Blocks (lập trình logic)

Hỗ trợ kiểm thử trực tiếp trên điện thoại qua app MIT AI2 Companion

Xuất file .apk để cài đặt trên Android

Hỗ trợ nhiều thành phần: nút bấm, nhãn, thanh trượt, camera, GPS, Bluetooth, WiFi...

Các Phần của MIT :

Designer : Kéo thả component, bố cục giao diện ứng dụng

Blocks : Lập trình logic bằng các khối lệnh màu sắc

AI2 Companion : Xem trước ứng dụng trực tiếp trên điện thoại

### **2.6.2. Tích hợp Firebase trong App Inventor**

App Inventor cung cấp component FirebaseDB tích hợp sẵn trong nhóm Storage, cho phép ứng dụng kết nối trực tiếp với Firebase Realtime Database mà không cần viết code HTTP/REST API thủ công. Chỉ cần cấu hình FirebaseURL và FirebaseToken, ứng dụng có thể đọc/ghi dữ liệu theo thời gian thực.

Trong đề tài này, App Inventor được dùng để xây dựng giao diện điều khiển đơn giản gồm nút Mở/Đóng khóa và nhãn hiển thị trạng thái relay, kết nối với node /LED\_PIN trên Firebase để gửi lệnh đến ESP32.

### **2.6.3. Thiết kế Blocks điều khiển**

Luồng hoạt động của ứng dụng được lập trình qua các khối lệnh (Blocks):

Khi nhấn nút MỞ KHÓA: `FirebaseDB.StoreValue(tag = " RELAY_KEY ", value = "1")`

Khi nhấn nút ĐÓNG KHÓA: `FirebaseDB.StoreValue(tag = " RELAY_KEY ", value = "0")`

Clock Timer (2 giây): `FirebaseDB.GetValue(tag = "RELAY_KEY")` để cập nhật trạng thái

Khi FirebaseDB.GotValue: cập nhật màu và text nhãn trạng thái tương ứng

## 2.7. Các Linh kiện phụ khác

### 2.7.1. Adapter 220V - 12V DC (Nguồn sơ cấp)

#### Đặc tính kỹ thuật:

Điện áp đầu vào: 100V - 240V AC (phù hợp với lưới điện Việt Nam).

Điện áp đầu ra: 12V DC cố định.

Dòng điện định mức: 3A (Công suất 36W).

Nguyên lý: Nguồn xung (Switching Mode Power Supply) tích hợp mạch lọc nhiễu EMI đầu vào.

#### Tại sao chọn:

Hiệu suất và Nhiệt lượng: So với biến áp sắt từ, Adapter nguồn xung có hiệu suất cao (>80%), tỏa nhiệt thấp, giúp tiết kiệm điện năng cho hệ thống chạy 24/7.

Phục vụ tải động lực: Khóa Solenoid cần đúng 12V để đạt lực hút tối đa. Việc chọn Adapter 12V giúp cấp điện trực tiếp cho khóa thông qua Relay mà không cần mạch tăng áp trung gian, giảm thiểu rủi ro hỏng hóc.

An toàn vận hành: Vỏ nhựa ABS cách điện hoàn toàn, bảo vệ bạn khỏi nguy cơ giật điện khi thao tác lắp ráp mô hình.

### 2.7.2. Mạch giảm áp LM2596S 3A



Hình 8 : Mạch giảm áp LM2596 4A

*Bảng 2 : Đặc tính kỹ thuật của module LM2596*

| Thông số           | Giá trị                        |
|--------------------|--------------------------------|
| Loại               | Buck converter (giảm áp DC-DC) |
| Điện áp vào        | 4V – 40V DC                    |
| Điện áp ra         | 1.23V – 35V DC (chỉnh được)    |
| Dòng ra tối đa     | 3A (có tản nhiệt: 4A)          |
| Hiệu suất          | Lên đến 92%                    |
| Tần số chuyển mạch | 150 kHz                        |
| Sụt áp tối thiểu   | 2V ( $V_{in} > V_{out} + 2V$ ) |
| Bảo vệ             | Ngắn mạch, quá nhiệt           |
| Nhiệt độ hoạt động | -40°C đến +85°C                |
| Chân IC            | 5 chân (TO-220 / TO-263)       |

Tính năng bảo vệ: Tự động ngắt khi quá nhiệt hoặc ngắn mạch đầu ra.

#### **Tại sao chọn:**

Độ tin cậy: So với các module hạ áp nhỏ (như AMS1117), LM2596 không bị nóng khi hạ áp từ 12V xuống 5V nhờ cơ chế băm xung PWM, giúp hệ thống hoạt động ổn định trong môi trường kín của hộp mô hình.

#### **2.7.3. XY-850 Mạch chuyển nguồn sạc dự phòng**



*Hình 9 : XY-850 Mạch chuyển nguồn sạc dự phòng*

#### **Đặc tính kỹ thuật:**

Chức năng: Tích hợp mạch sạc pin Lithium và mạch chuyển đổi nguồn tự động (Automatic Power Switching).

Điện áp chuyển đổi: Tương thích hệ pin 3.7V - 4.2V.

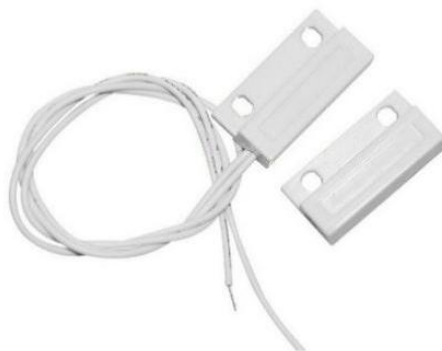
Cơ chế: Chuyển sang nguồn pin ngay lập tức khi mất điện lưới mà không làm đứt quãng dòng điện.

**Tại sao chọn:**

Tính liên tục (Non-stop): Đây là "trái tim" giúp đồ án của bạn trở thành một hệ thống an ninh chuyên nghiệp. Khi kẻ gian cắt điện lưới, XY-850 đảm bảo STM32 và ESP32 vẫn hoạt động thông qua nguồn phụ.

Quản lý pin thông minh: Tích hợp sẵn chức năng bảo vệ pin khỏi việc xả cạn (gây hỏng pin) và sạc quá đầy (gây cháy nổ), giúp hệ thống an toàn khi sử dụng lâu dài.

**2.7.4. Cảm biến từ MC-38**



*Hình 10 : Cảm biến từ MC-38*

**Đặc tính kỹ thuật:**

Cấu tạo: Tiếp điểm cơ khí lưỡi gà (Reed Switch) và nam châm vĩnh cửu.

Khoảng cách tác động: 15mm - 25mm.

Trạng thái: Thường đóng (NC - khi có nam châm).

**Tại sao chọn:**

Độ tin cậy tuyệt đối: Không giống như cảm biến hồng ngoại (dễ bị nhiễu bởi ánh nắng) hay cảm biến siêu âm (dễ báo giả do vật cản), MC-38 chỉ hoạt động dựa trên từ trường vật lý, cho kết quả chính xác 100% về trạng thái cửa.

Tiết kiệm năng lượng: Cảm biến không tiêu thụ điện ở trạng thái chờ, điều này cực kỳ quan trọng khi hệ thống đang chạy bằng pin dự phòng, giúp kéo dài thời gian hoạt động của toàn mô hình.

#### 2.7.5. Cell pin 12V (Nguồn dự phòng)



Hình 11 : Cell pin 12V

##### Cấu tạo:

Pin 18650 là pin Li-ion hình trụ, kích thước 18mm × 65mm. Gói pin 12V gồm 3 cell nối tiếp (3S):

$$\begin{aligned} \text{Cell 1 (3.7V)} + \text{Cell 2 (3.7V)} + \text{Cell 3 (3.7V)} &= 11.1\text{V nominal} \\ &= 12.6\text{V khi đầy} \\ &= 9\text{V khi cạn} \end{aligned}$$

Mỗi cell có thể ghép song song thêm để tăng dung lượng (3S2P, 3S3P...).

Bảng 3 : Thông số kỹ thuật của cell pin

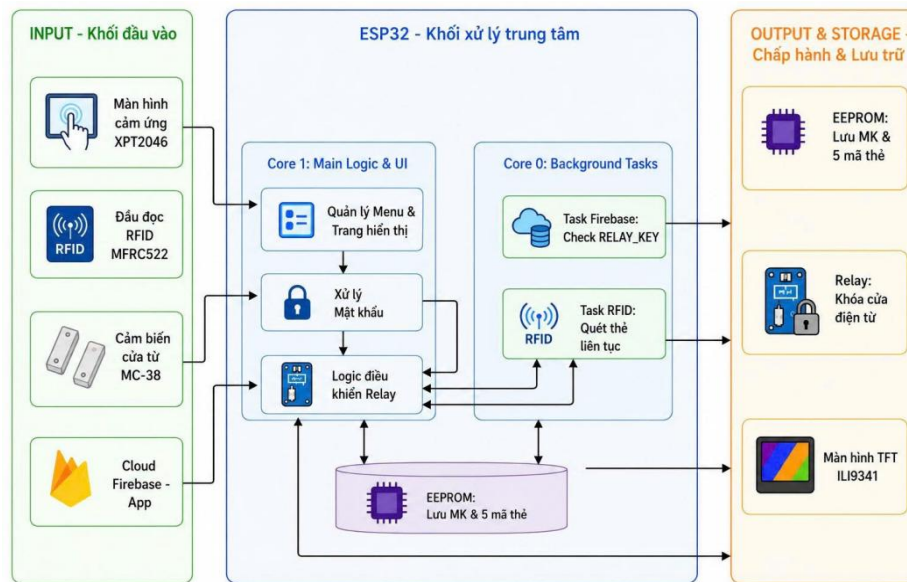
| Thông số            | Giá trị                            |
|---------------------|------------------------------------|
| Điện áp danh định   | 11.1V (ghi là "12V")               |
| Điện áp đầy         | 12.6V                              |
| Điện áp cạn         | 9V                                 |
| Dung lượng phổ biến | 4400mAh – 20000mAh                 |
| Hóa học             | Li-ion (Lithium Ion)               |
| Dòng xả tối đa      | 2C – 5C tùy cell                   |
| Có BMS tích hợp     | Bảo vệ quá áp, quá dòng, ngắn mạch |

Thông số giá trị điện áp danh định 11.1V (ghi là "12V"). Điện áp đầy 12.6V. Điện áp cạn 9V. Dung lượng phổ biến 4400mAh – 20000 mAh Hóa học Li-ion (Lithium Ion). Dòng

xả tối đa 2C – 5C tùy cell có BMS tích hợp (bảo vệ quá áp, quá dòng, ngắn mạch). Cell pin có trang bị sẵn jack cắm output và jack cắm sạc.

## CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

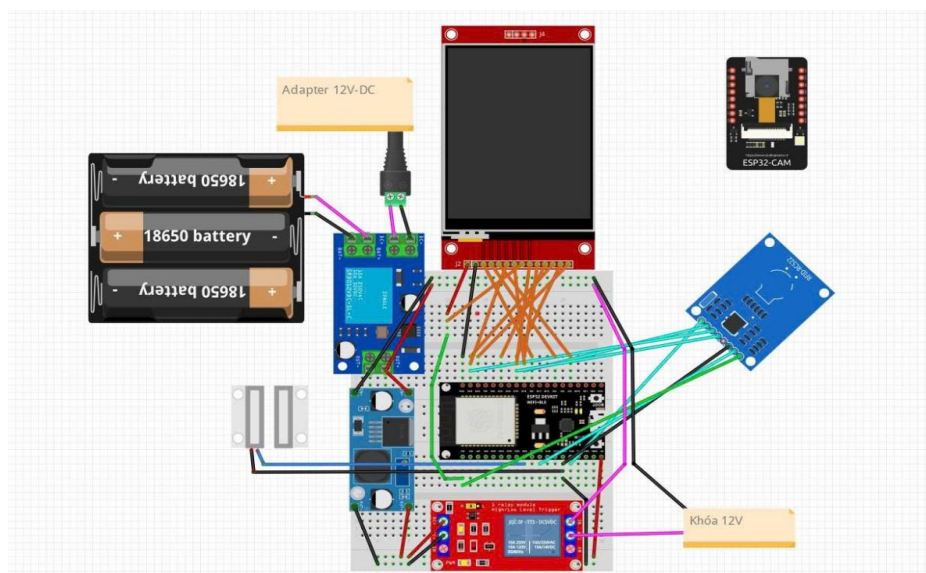
### 3.1. Sơ đồ khối tổng thể



Hình 12 : Sơ đồ khối tổng thể

### 3.2. Sơ đồ mạch nguyên lý và kết nối dây

Sơ đồ mạch nguyên lý mô phỏng:



Hình 13 : Sơ đồ mạch nguyên lý

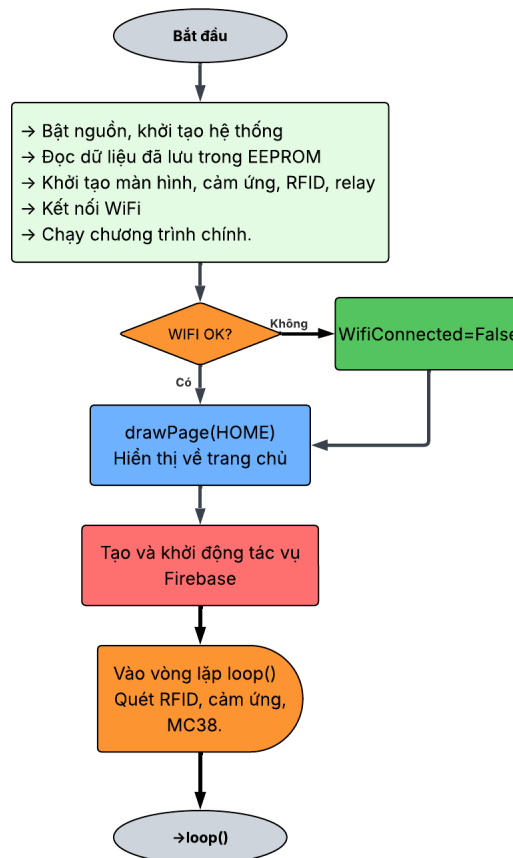


Bảng 4 : Kết nối dây của mô hình

| GPIO | Kết Nối         | Mô Tả                           |
|------|-----------------|---------------------------------|
| 18   | SCK (SPI Clock) | Chung: TFT + Touch + RFID       |
| 19   | MISO (SPI)      | Chung: TFT + Touch + RFID       |
| 23   | MOSI (SPI)      | Chung: TFT + Touch + RFID       |
| 5    | TFT_CS          | Chip Select màn hình ILI9341    |
| 2    | TFT_DC          | Data/Command màn hình           |
| 4    | TFT_RST         | Reset màn hình                  |
| 21   | TOUCH_CS        | Chip Select cảm ứng XPT2046     |
| 13   | RFID_SS         | Chip Select module RC522        |
| 27   | RFID_RST        | Reset module RC522              |
| 33   | RELAY_PIN       | Điều khiển relay → Solenoid 12V |
| 25   | MC38_PIN        | Cảm biến cửa MC-38              |
| GND  | GND chung       | TFT, RFID, Relay, MC-38         |

### 3.3. Lưu đồ thuật toán

#### 3.3.1. Khởi động



Hình 14 : Lưu đồ khởi động

## **Mô tả lưu đồ quá trình khởi động hệ thống (Setup):**

### *Bắt đầu:*

Khi cấp điện cho ESP32, thiết bị sẽ tự động chạy đoạn code khởi động đầu tiên giống như việc bật máy tính lên và chờ nó nạp xong hệ điều hành.

### *Bước "Khởi động các bộ phận" (khối xanh lá):*

Đây là lúc thiết bị chuẩn bị mọi thứ trước khi hoạt động, làm lần lượt các việc sau:

Mở cổng debug: bật cổng Serial để nếu cắm máy tính vào, mình có thể xem được log hoạt động của thiết bị.

Lấy lại thông tin đã lưu trước đó: đọc từ bộ nhớ EEPROM (bộ nhớ không mất dữ liệu khi cúp điện) để lấy lại mật khẩu người dùng đã đặt và danh sách các thẻ RFID đã đăng ký trước đó nhờ vậy dù mất điện, thiết bị vẫn "nhớ" mật khẩu và thẻ cũ chứ không bị reset về mặc định. Thiết bị còn tự kiểm tra xem dữ liệu đọc ra có "hợp lệ" không (có phải toàn ký tự chữ/số bình thường không) nếu bộ nhớ bị lỗi hoặc chưa từng ghi gì (máy mới), nó sẽ tự đặt lại mật khẩu về "1234" cho chắc ăn, tránh trường hợp không mở được cửa vì đọc phải dữ liệu rác.

Chuẩn bị phần cứng: bật relay (khóa cửa) ở trạng thái đóng an toàn ngay từ đầu để nếu có sự cố khi khởi động, cửa mặc định vẫn khóa chứ không tự mở toang. Đồng thời đọc cảm biến từ trên cửa, và dọn sẵn các đường giao tiếp cho màn hình, cảm ứng, đầu đọc thẻ để chúng không "giành nhau" khi cùng hoạt động, vì cả ba thiết bị này dùng chung một đường truyền dữ liệu (bus SPI) nên phải "nhường nhau" theo thứ tự.

Bật màn hình, cảm ứng, đầu đọc thẻ: giống như bật màn hình điện thoại và kiểm tra cảm ứng có ăn không, đồng thời kiểm tra đầu đọc thẻ RFID có được nhận diện đúng hay không (thiết bị đọc một mã phiên bản từ chip đọc thẻ, nếu đúng mã quen thuộc thì yên tâm là module RFID đang gắn đúng và hoạt động tốt).

Kết nối WiFi: thiết bị thử kết nối vào mạng WiFi đã cấu hình sẵn, cố gắng thử trong khoảng 10 giây (chia làm 20 lần thử nhỏ, mỗi lần cách nhau nửa giây). Nếu WiFi bắt được nhanh thì đi tiếp luôn, không cần chờ hết 10 giây; còn nếu quá 10 giây vẫn chưa bắt được thì thiết bị bỏ qua, không chờ mãi làm treo máy.

*Kiểm tra "WiFi có kết nối được không?" (hình thoi):*

Đây là bước rẽ nhánh:

Nếu không kết nối được: thiết bị vẫn hoạt động bình thường, chỉ là tính năng mở cửa qua app điện thoại (qua Firebase) sẽ chưa dùng được lúc này. Mở cửa bằng mật khẩu hoặc bằng thẻ vẫn dùng tốt. Thiết bị cũng không cố kết nối lại WiFi ngay lập tức, mà để dành việc đó - nếu sau này người dùng muốn, có thể cắm điện lại để thử kết nối từ đầu.

Nếu kết nối được: thiết bị ghi nhận đã có mạng và sẵn sàng dùng tính năng mở cửa từ xa qua app, đồng thời in ra địa chỉ mạng nội bộ của mình để tiện kiểm tra nếu cần.

Dù đi nhánh nào, sau đó thiết bị cũng tiếp tục sang bước tiếp theo như nhau.

*Hiện màn hình chính (khối xanh dương):*

Thiết bị vẽ lên màn hình giao diện chính: 3 nút to để chọn cách mở cửa (mật khẩu / qua app / tùy chỉnh), dòng chữ nhắc quét thẻ, và một dòng nhỏ phía dưới cho biết cửa đang khóa hay mở, có mạng hay không. Từ lúc này, người dùng có thể chạm tay vào các nút này để chuyển sang các màn hình chức năng khác.

Bật tính năng điều khiển từ xa (khối đỏ)

Thiết bị tạo ra một "luồng xử lý riêng" chạy ngầm song song, chuyên đi hỏi máy chủ Firebase mỗi 2 giây một lần xem có lệnh mở cửa từ app gửi tới không. Việc này chạy độc lập, không làm chậm hay ảnh hưởng tới việc thiết bị xử lý màn hình, cảm ứng ở phía trước - giống như một chiếc điện thoại vừa nghe nhạc vừa lướt web mà không bị giật, vì hai việc được giao cho hai "bộ não" (lõi CPU) khác nhau xử lý riêng.

*Vào vòng lặp chính (khối cam):*

Sau khi khởi động xong, thiết bị bước vào chế độ hoạt động thường trực, liên tục lặp đi lặp lại các việc sau, không nghỉ. Cứ khoảng 0.3 giây lại kiểm tra xem có ai quét thẻ RFID không, thẻ đúng thì mở cửa, thẻ lạ thì báo sai.

Theo dõi cảm biến cửa để biết cửa đã đóng lại chưa, nếu đóng rồi thì tự động khóa ngay - thiết bị còn đọc cảm biến hai lần liên tiếp cách nhau rất ngắn để chắc chắn không bị nhiễu tín hiệu làm khóa nhầm.

Nếu cửa mở quá 10 giây mà chưa ai đóng lại thì tự động khóa cho an toàn, phòng trường hợp người dùng quên đóng cửa.

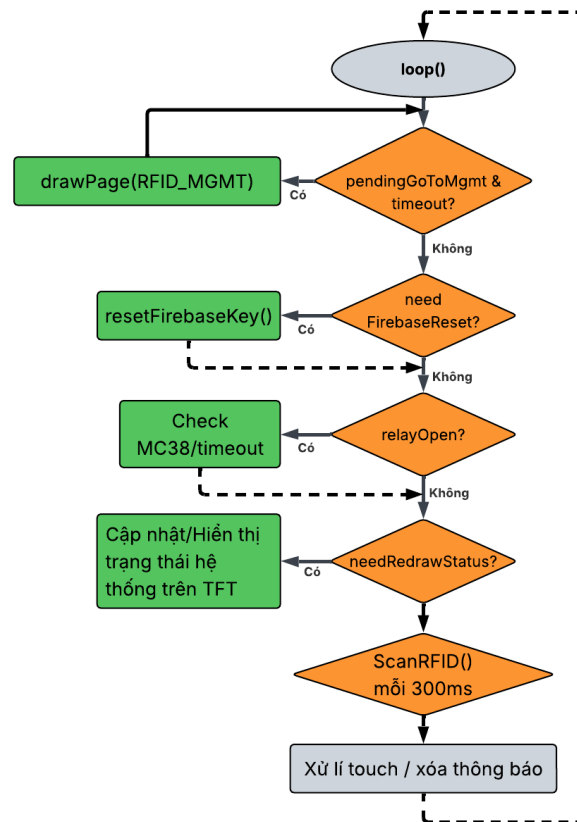
Xử lý khi người dùng chạm vào màn hình (bấm số, chuyển trang...), bằng cách đổi tọa độ chạm tay thành đúng vị trí trên màn hình rồi xem người dùng đang bấm vào nút hay phím nào.

Tự động ẩn các thông báo (như "sai mật khẩu") sau khoảng gần 2 giây hiển thị, để màn hình không bị rối vì thông báo cũ còn đọng lại.

*Kết thúc → quay lại vòng lặp:*

Vòng lặp này không bao giờ dừng nó chính là "nhịp tim" của thiết bị, cứ chạy mãi cho tới khi bị cúp điện hoặc bị reset lại.

### 3.3.2. Vòng lặp xử lý các tác vụ



Hình 15 : Lưu đồ thuật toán vòng lặp xử lý

#### Mô tả lưu đồ vòng lặp xử lý chính:

*Ý tưởng cốt lõi:*

loop() trong Arduino chạy đi chạy lại liên tục, hàng nghìn lần mỗi giây. Lưu đồ này mô tả một vòng chạy (một lần đi từ trên xuống dưới). Mỗi vòng, ESP32 sẽ lần lượt hỏi 5 câu hỏi

"Có việc gì cần làm không?" nếu có thì làm, xong lại hỏi câu tiếp theo, hết 5 câu thì quay lại từ đầu.

Điểm mấu chốt cần nắm: chỉ có bước đầu tiên là "thoát sớm", còn 4 bước sau dù trả lời Có hay Không thì vẫn đi tiếp xuống dưới, không bỏ cuộc giữa chừng.

Đi từng bước:

*Bước 1 - "Vừa thêm thẻ RFID xong, đã đủ 2 giây chưa?"*

(điều kiện: pendingGoToMgmt && millis()-addCardTimer>2000)

Đây là tình huống: vừa quét thẻ mới ở trang "Thêm thẻ", màn hình đang hiện chữ "Thêm thành công" và cần đợi 2 giây cho người dùng đọc xong trước khi tự động chuyển về trang danh sách thẻ.

Có (đã đủ 2s) → gọi drawPage(RFID\_MGMT) để vẽ lại màn hình danh sách thẻ, rồi dừng luôn, không hỏi tiếp 4 câu bên dưới, chờ vòng lặp kế tiếp mới hỏi lại từ đầu.

Không (chưa đủ 2s, hoặc không trong tình huống này) → bỏ qua, đi xuống câu 2.

*Bước 2 - "Có cần reset khóa Firebase không?"*

(điều kiện: needFirebaseReset == true)

Tình huống: cửa vừa được mở bằng App điện thoại (qua Firebase), rồi cửa cũng vừa đóng lại hoặc hết giờ tự khóa. Lúc này ESP32 cần báo ngược lại cho Firebase "tôi đã khóa cửa rồi, ông đặt cờ RELAY\_KEY về 0 đi" để lần sau App không tưởng nhầm là cửa vẫn đang mở.

Có → gọi resetFirebaseKey(), gửi lệnh PUT lên Firebase set giá trị về 0.

Không → bỏ qua.

Dù Có hay Không, vẫn đi tiếp xuống câu 3 (khác bước 1).

*Bước 3 - "Relay (khóa cửa) hiện đang mở không?"*

(điều kiện: relayOpen == true)

Có → chạy toàn bộ khối kiểm tra cảm biến cửa MC38 và đếm giờ timeout đây chính là cái lưu đồ chi tiết mình vẽ ở lượt trước (đọc cảm biến 2 lần chống dội tín hiệu, xem cửa vừa mở rồi đóng lại chưa, hoặc đã quá 10 giây chưa để tự khóa lại).

Không (cửa đang khóa sẵn, chẳng có gì để theo dõi) → bỏ qua toàn bộ khối đó.

Dù Có hay Không, vẫn đi tiếp xuống bước 4.

*Bước 4 - "Có cần vẽ lại trạng thái lên màn hình không?"*

(điều kiện: needRedrawStatus == true)

Cờ này được bật lên ở nhiều chỗ khác nhau trong code (vừa mở cửa xong, vừa đóng cửa xong, vừa quét thẻ RFID xong...). Đây là bước "dọn dẹp": kiểm tra xem có cờ báo hiệu nào đang chờ vẽ lại màn hình không.

Có → gọi hàm cập nhật hiển thị (dòng chữ "Đang mở cửa", tên WiFi, thông báo RFID...) tùy đang ở trang nào.

Không → bỏ qua.

Vẫn đi tiếp xuống bước 5.

*Bước 5 - "Quét thẻ RFID"*

Đây không phải câu hỏi Có/Không, mà là hành động định kỳ: cứ mỗi 300ms trôi qua thì thực hiện quét thẻ một lần (kiểm tra có thẻ nào áp vào đầu đọc không). Vì đặt trong loop() chạy liên tục nên việc quét thẻ diễn ra song song, không làm gián đoạn các thao tác chạm màn hình.

*Bước 6 - "Xử lý cảm ứng / xóa thông báo cũ"*

Bước cuối cùng, gộp 3 việc nhỏ:

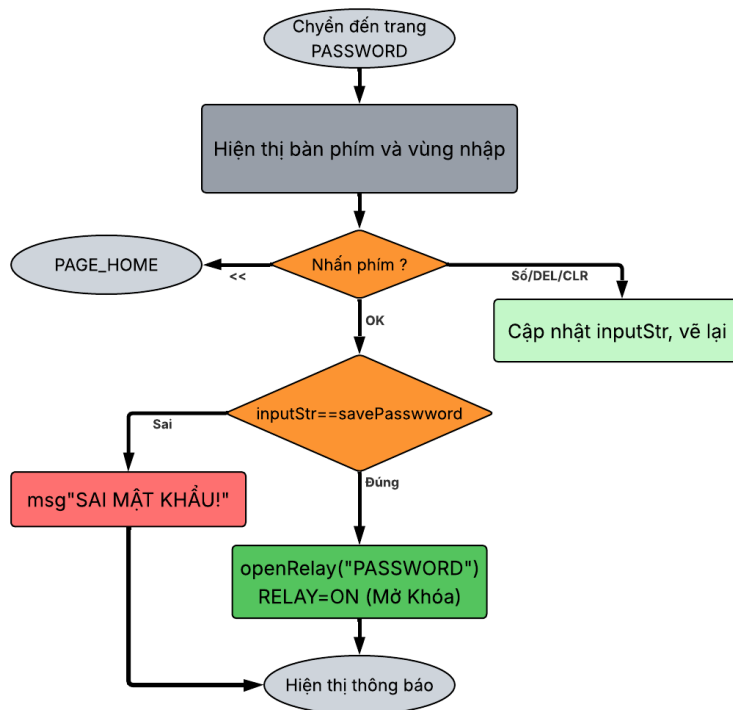
Xóa các dòng chữ thông báo (như "Sai mật khẩu!", "Đã hết hạn thẻ...") nếu đã hiện quá lâu.

Đọc xem người dùng có đang chạm vào màn hình không, ở tọa độ nào.

Nếu có chạm, xử lý tùy theo đang ở trang nào (trang chủ, bàn phím, quản lý thẻ...).

Xong bước 6, mũi tên nét đứt quay thẳng về đỉnh loop(), cả vòng lặp lại bắt đầu từ Bước 1.

### **3.3.3. Nhập mật khẩu**



Hình 16 : Lưu đồ thuật toán phương thức nhập MẬT KHẨU

### Mô tả lưu đồ nhập mật khẩu:

Bắt đầu "Chuyển đến trang PASSWORD":

Khi người dùng ở màn hình chính chạm vào nút "MẬT KHẨU", thiết bị gọi `drawPage(PAGE_PASSWORD)`. Hàm này đầu tiên reset các biến trạng thái trang về rỗng (`inputStr`, `msgText`...), xóa toàn bộ màn hình về màu nền, rồi mới bắt đầu vẽ giao diện trang mật khẩu.

Khối "Hiện thị bàn phím và vùng nhập" (xám):

Thiết bị vẽ hai phần: một ô hiển thị phía trên với viền màu xanh dương nhạt (viền sẽ chuyển xanh lá nếu cửa đang mở bằng mật khẩu), ban đầu ghi chữ mờ "Nhập mật khẩu..."; và một bàn phím số 4x4 gồm 16 phím: các số 0-9, ký tự \*, #, cùng 4 phím chức năng ở cột cuối - DEL (xóa từng ký tự), CLR (xóa hết), OK (xác nhận), << (quay lại). Mỗi phím có vùng tọa độ chạm riêng đã được tính sẵn. Từ đây thiết bị vào trạng thái chờ, liên tục kiểm tra cảm ứng màn hình ở vòng lặp `loop()` chính.

Kiểm tra "Nhấn phím?" (hình thoi):

Khi phát hiện có chạm tay, thiết bị tính xem tọa độ chạm rơi vào phím nào trong lưới 4x4, rồi xử lý theo 3 nhánh:

<< → gọi thẳng `drawPage(PAGE_HOME)`, thoát khỏi trang mật khẩu và quay về màn hình chính ngay lập tức, không cần xử lý gì thêm, không lưu lại gì đã gõ dở.

Số / DEL / CLR → đi sang khối xanh nhạt "Cập nhật `inputStr`, vẽ lại": nếu là phím số thì nối thêm ký tự đó vào cuối chuỗi `inputStr` (miễn là chưa vượt quá 8 ký tự giới hạn độ dài mật khẩu tối đa); nếu là DEL thì cắt bỏ ký tự cuối cùng của `inputStr`; nếu là CLR thì xóa trắng toàn bộ `inputStr` về rỗng. Sau mỗi lần bấm, thiết bị vẽ lại ô hiển thị: số ký tự đã gõ được thay bằng các dấu \* để che mật khẩu thật, đồng thời phím vừa bấm cũng đổi màu sáng lên trong chốc lát để phản hồi cảm ứng (nhấn nhả). Sau khi vẽ xong, luồng xử lý quay ngược lại đúng bước "Nhấn phím?" phía trên để chờ lượt chạm kế tiếp người dùng có thể gõ bao nhiêu lần tùy ý, vòng lặp này không tự thoát.

OK → thiết bị coi như người dùng đã gõ xong mật khẩu định thử, gọi hàm `checkPassword()` và đi tiếp xuống bước so sánh.

*Kiểm tra "inputStr == savedPassword" (hình thoi):*

Hàm `checkPassword()` so sánh chuỗi vừa gõ với mật khẩu đã lưu trong bộ nhớ (biến `savedPassword`, được nạp từ EEPROM lúc khởi động): trước tiên so độ dài hai chuỗi có bằng nhau không, nếu bằng thì so tiếp từng ký tự một; chỉ cần lệch một ký tự là coi như sai ngay:

Sai → đặt biến thông báo `msgText = "SAI MAT KHAU!"` với màu đỏ, ghi lại thời điểm hiện tại để tự đếm giờ âm thông báo, đồng thời xóa trắng `inputStr` để người dùng phải gõ lại từ đầu (không giữ lại phần gõ sai). Khối "msg SAI MẬT KHẨU!" màu đỏ được kích hoạt.

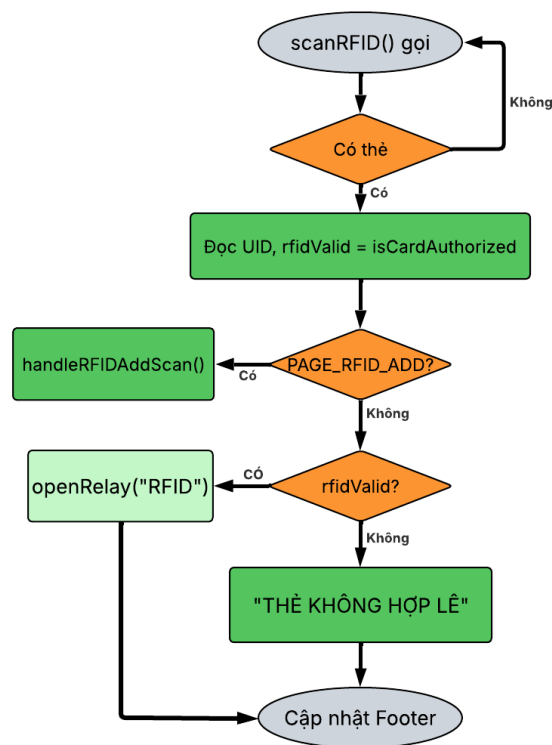
Đúng → gọi `openRelay("PASSWORD")`: chân relay được đưa lên mức bật (ON) để mở khóa cửa thật sự bằng phần cứng, đồng thời ghi nhận `openSource = "PASSWORD"` (biến này dùng để sau này hiển thị "CUA DANG MO: PASSWORD" trên thanh trạng thái, và để phân biệt nguồn mở cửa khi tự động khóa lại). Bộ đếm thời gian mở cửa (`relayTimer`) cũng được đặt lại từ 0 để chuẩn bị cho việc tự khóa sau 10 giây. Thông báo `msgText = "MO CUA THANH CONG!"` màu xanh lá được chuẩn bị.



Khởi "Hiển thị thông báo" (kết thúc):

Dù đúng hay sai, cả hai nhánh đều hội tụ về đây: ô hiển thị và thanh trạng thái bên dưới được vẽ lại ngay lập tức để hiển thị thông báo tương ứng (đỏ nếu sai, xanh nếu thành công). Thông báo này không tự động biến mất ngay mà được canh giờ ở vòng lặp loop() chính có đoạn riêng kiểm tra: hễ quá 1.8 giây kể từ lúc hiển thị thông báo thì tự xóa msgText và vẽ lại ô nhập về trạng thái rỗng ban đầu, sẵn sàng cho lượt nhập tiếp theo. Nếu mật khẩu đúng và cửa đã mở, việc theo dõi cảm biến cửa để tự khóa lại (khi cửa đóng) hoặc khóa cưỡng bức sau 10 giây timeout được xử lý ở một nhánh khác của loop() chính, nằm ngoài phạm vi lưu đồ nhập mật khẩu này nghĩa là người dùng có thể rời khỏi trang mật khẩu (bấm << quay về Home) ngay sau khi mở cửa mà việc tự khóa vẫn diễn ra bình thường ở nền.

### 3.3.4. Quét thẻ Rfid



Hình 17 : Lưu đồ thuật toán phương thức thẻ Rfid

#### Mô tả lưu đồ nhập thẻ Rfid:

Bắt đầu "scanRFID() gọi":

Đây là hàm được gọi liên tục trong loop() chính, cứ mỗi 300ms một lần (kiểm tra bằng biến đếm thời gian lastRFIDScan), bất kể người dùng đang ở trang nào trên màn Hình nó

chạy ngầm song song với việc xử lý cảm ứng, để lúc nào cũng sẵn sàng nhận diện thẻ mà không cần người dùng phải vào một màn hình riêng để quét thẻ.

*Kiểm tra "Có thẻ" (hình thoi):*

Thiết bị gọi `mfr522.PICC_IsNewCardPresent()` để hỏi module đọc thẻ RC522 xem có thẻ nào đang áp gần đầu đọc không:

Không → tăng một bộ đếm số lần "quét hụt" (`rfidFailCount`) lên 1, rồi quay ngược lại đầu vòng lặp, coi như chưa có gì xảy ra, chờ tới lượt quét kế tiếp (300ms sau). Nếu tình trạng "không thấy thẻ" này lặp lại liên tục đủ 10 lần (~3 giây liên không quét được gì), thiết bị sẽ tự động gọi lại `mfr522.PCD_Init()` để khởi động lại module RFID một lần, phòng trường hợp module bị treo, kẹt giao tiếp SPI, hoặc bị nhiễu đây là một cơ chế "tự hồi phục" để không cần phải tắt mở nguồn thủ công nếu đầu đọc bị đơ.

Có → gọi tiếp `mfr522.PICC_ReadCardSerial()` để đọc dữ liệu thẻ; nếu bước đọc này thất bại (thẻ nhiễu, quét dở dang) thì thiết bị cũng tự `PCD_Init()` lại và bỏ qua lượt này. Nếu đọc thành công, bộ đếm `rfidFailCount` được reset về 0 và đi tiếp xuống bước đọc UID.

*Khởi "Đọc UID, `rfidValid = isCardAuthorized`" (xanh lá đậm):*

Thiết bị duyệt qua từng byte trong mã UID của thẻ, ghép lại thành một chuỗi hex (ví dụ "3A 5F 12 8B"), viết hoa toàn bộ để đồng nhất định dạng. Chuỗi UID này được lưu vào biến `rfidUID`, đồng thời gọi hàm `isCardAuthorized()` để so sánh với từng thẻ trong danh sách đã lưu (`cardList`, tối đa 5 thẻ) nếu trùng khớp với bất kỳ thẻ nào thì `rfidValid = true`. Sau khi đọc xong, thiết bị gọi `PICC_HaltA()` và `PCD_StopCrypto1()` để "ngắt kết nối" tạm thời với thẻ, tránh việc đọc lặp lại liên tục cùng một thẻ trong các lượt quét kế tiếp khi thẻ vẫn còn áp gần đầu đọc. Toàn bộ UID và kết quả hợp lệ/không cũng được in ra Serial để tiện debug.

*Kiểm tra "PAGE\_RFID\_ADD?" (hình thoi):*

Thiết bị xem biến `currentPage` hiện tại có đang bằng `PAGE_RFID_ADD` hay không (tức người dùng đang ở màn hình "Thêm thẻ mới" trong phần quản lý) vì cùng một thao tác quét thẻ nhưng ý nghĩa hoàn toàn khác nhau tùy ngữ cảnh:

Có → đi sang khối `handleRFIDAddScan()`: đây là lúc admin đang trong quy trình đăng ký thẻ mới. Thẻ vừa quét được kiểm tra xem đã tồn tại trong danh sách chưa (tránh thêm

trùng); nếu chưa có và còn chỗ trống (dưới 5 thẻ) thì được addCard() ghi thêm vào danh sách và lưu xuống EEPROM để không mất khi cúp điện; nếu đã tồn tại thì báo "THE DA TON TAI!". Thẻ ở nhánh này không dùng để mở cửa, dù thẻ đó có hợp lệ hay không.

Không → nghĩa là đây là một lượt quét thẻ bình thường ở bất kỳ trang nào khác (kể cả màn hình chính), đi tiếp xuống bước kiểm tra thẻ có hợp lệ không để quyết định mở cửa.

*Kiểm tra "rfidValid?" (hình thoi):*

Đây là bước quyết định thẻ vừa quét (ở ngữ cảnh mở cửa bình thường) có được phép mở cửa hay không, dựa trên kết quả đã tính ở bước đọc UID phía trên:

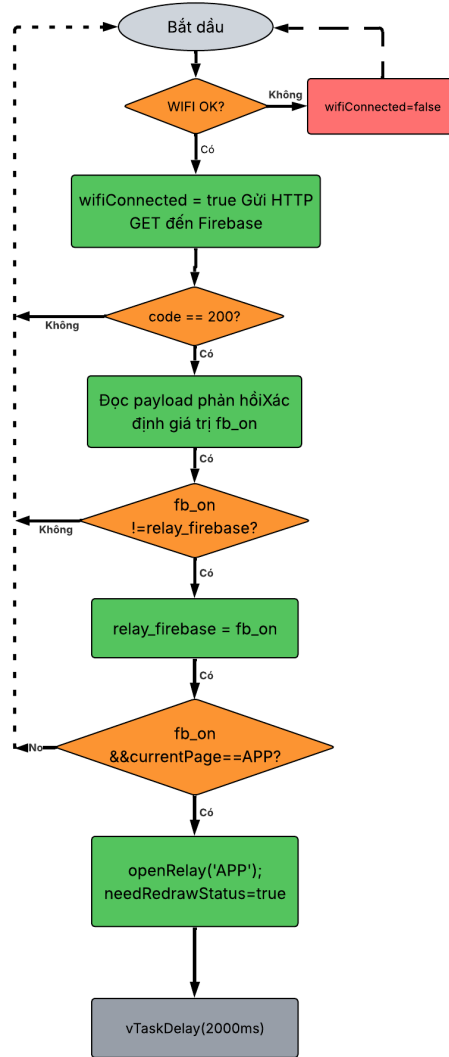
Có → đi sang khối xanh nhạt openRelay("RFID"): thiết bị bật relay lên mức ON để mở khóa cửa thật sự, ghi nhận nguồn mở cửa là "RFID" (dùng để hiển thị trạng thái ">>> CUA DANG MO: RFID" trên thanh trạng thái, và để phân biệt với việc mở bằng mật khẩu hay app khi tự động khóa lại sau này), đồng thời chuẩn bị thông báo "THE HOP LE - MO CUA!" màu xanh lá.

Không → đi sang khối xanh lá "THẺ KHÔNG HỢP LỆ": thiết bị chỉ chuẩn bị thông báo lỗi màu đỏ "THE KHONG HOP LE!" để hiển thị, không tác động gì tới relay hay trạng thái cửa.

*Khối "Cập nhật Footer" (kết thúc):*

Dù thẻ hợp lệ hay không, cả hai nhánh đều hội tụ về đây tại hàm refreshRFIDMsgArea(): thiết bị kiểm tra đang ở trang nào để vẽ thông báo đúng chỗ - nếu đang ở trang chủ thì vẽ vào drawStatusFooter() (thanh cuối màn hình), còn nếu đang ở các trang nhập liệu khác (mật khẩu, app, đổi mật khẩu, xác nhận quyền) thì vẽ vào drawBackBar() (thanh phía trên nút BACK). Thông báo được ghi thời điểm hiện tại vào rfidMsgTimer, và ở vòng lặp loop() chính có một đoạn riêng theo dõi: hễ quá 1.8 giây kể từ lúc hiện thông báo thì tự động xóa nó đi và vẽ lại thanh trạng thái về trạng thái bình thường. Sau bước này, toàn bộ quy trình quay lại chờ 300ms để thực hiện lượt quét tiếp theo, lặp lại vô thời hạn song song với mọi hoạt động khác của thiết bị.

### **3.3.5. Trang APP/FIREBASE (PAGE\_APP)**



Hình 18 : Lưu đồ thuật toán phương thức APP

### Mô tả lưu đồ APP Firebase (PAGE\_APP):

*Bắt đầu (vòng lặp firebaseTask):*

Đây chính là hàm firebaseTask() - một luồng xử lý (task) riêng biệt được tạo ra ngay từ setup() bằng xTaskCreatePinnedToCore(), chạy cố định trên lõi Core 0 của ESP32, hoàn toàn tách biệt và song song với vòng lặp loop() chính (chạy trên Core 1 để xử lý màn hình, cảm ứng, RFID). Bên trong là một vòng while(true) chạy mãi mãi, cứ lặp đi lặp lại toàn bộ các bước dưới đây - đó là lý do có mũi tên nét đứt vòng ngược lại "Bắt đầu" ở cuối lưu đồ.

*Kiểm tra "WIFI OK?" (hình thời):*

Mỗi vòng lặp, thiết bị kiểm tra `WiFi.status() == WL_CONNECTED`:

Không → đi sang khối đỏ wifiConnected = false: chỉ đơn giản ghi nhận là hiện không có mạng, rồi bỏ qua toàn bộ các bước còn lại, quay thẳng về đầu vòng lặp (đường nét đứt bên phải chạy vòng lên "Bắt đầu"). Việc gọi Firebase sẽ không được thử trong vòng này nữa, đợi tới vòng sau.

Có → đi sang khối xanh lá "wifiConnected = true. Gửi HTTP GET đến Firebase": ghi nhận đang có mạng, sau đó mở kết nối HTTP tới địa chỉ RELAY\_KEY.json trên Firebase Realtime Database, đặt thời gian chờ tối đa (timeout) là 2 giây, rồi gửi yêu cầu GET để hỏi giá trị hiện tại của RELAY\_KEY - đây chính là "hộp thư" mà app điện thoại sẽ ghi số 1 vào khi muốn mở cửa từ xa.

*Kiểm tra "code == 200?" (hình thoi):*

Thiết bị xem mã phản hồi HTTP trả về từ Firebase:

Không (khác 200, ví dụ mất mạng giữa chừng, lỗi máy chủ, timeout...) → coi như lần hỏi này thất bại, bỏ qua, đóng kết nối HTTP và quay về đầu vòng lặp, không đọc dữ liệu gì thêm.

Có (đúng 200 - nghĩa là Firebase phản hồi thành công) → đi tiếp xuống bước đọc nội dung trả về.

*Khối "Đọc payload phản hồi. Xác định giá trị fb\_on" (xanh lá):*

Thiết bị đọc toàn bộ nội dung phản hồi (http.getString()) đây chính là giá trị hiện tại của RELAY\_KEY, ví dụ "1", "0" hoặc "1" có ngoặc kép tùy định dạng JSON Firebase trả về. Chuỗi này được cắt bỏ khoảng trắng thừa (trim()), sau đó thiết bị kiểm tra xem nó có bằng "1" hoặc "\"1\"" hay không để xác định biến fb\_on là true (app đang yêu cầu mở cửa) hay false (không có yêu cầu gì).

Kiểm tra "fb\_on != relay\_firebase?" (hình thoi)

Đây là bước so sánh giá trị vừa đọc được (fb\_on) với giá trị đã ghi nhận từ lần hỏi trước đó (relay\_firebase, biến toàn cục lưu trạng thái):

Không (giá trị không đổi so với lần trước ví dụ vẫn đang là "0" như cũ, hoặc vẫn đang là "1" như cũ) → nghĩa là không có gì mới cần xử lý, quay thẳng về đầu vòng lặp, tránh việc cứ mỗi 2 giây lại mở relay lặp lại dù app không gửi lệnh mới.

Có (giá trị vừa đọc khác với lần trước tức Firebase vừa được app cập nhật) → đi tiếp, vì đây là một sự kiện thay đổi thật sự cần xử lý.

*Khối "relay\_firebase = fb\_on" (xanh lá):*

Thiết bị cập nhật lại biến toàn cục relay\_firebase bằng giá trị fb\_on vừa đọc, để lần hỏi kế tiếp (2 giây sau) có cái để so sánh và biết được có gì thay đổi hay không.

*Kiểm tra "fb\_on && currentPage == APP?" (hình thoi):*

Đây là điều kiện kép quyết định có thật sự mở cửa hay không:

No (một trong hai điều kiện sai hoặc fb\_on là false tức app không yêu cầu mở, hoặc app có yêu cầu mở nhưng người dùng hiện không đang đứng ở trang APP trên màn hình) → quay về đầu vòng lặp, không mở relay. Đây là một cơ chế an toàn có chủ đích: dù app gửi lệnh mở cửa, thiết bị chỉ thật sự mở khóa nếu người đứng tại chỗ cũng đang mở đúng màn hình APP để theo dõi, tránh trường hợp cửa tự mở bất ngờ khi không ai để ý màn hình.

(Cả hai điều kiện đúng) → đi tiếp xuống bước mở cửa thật sự.

*Khối "openRelay('APP'); needRedrawStatus = true" (xanh lá):*

Thiết bị gọi openRelay("APP"): bật chân relay lên mức ON để mở khóa cửa thật sự bằng phần cứng, ghi nhận nguồn mở cửa là "APP" (dùng để sau này hiển thị ">>> MÔ CUA <<<" trên trang APP, và để nhận biết cần tự động ghi giá trị RELAY\_KEY về lại "0" trên Firebase sau khi cửa đóng lại). Đồng thời đặt cờ needRedrawStatus = true cờ này được vòng lặp loop() chính (chạy ở Core 1) đọc và dùng để vẽ lại giao diện trên màn hình, vì bản thân task Firebase này chạy ở Core 0 nên không trực tiếp đụng vào việc vẽ màn hình mà chỉ "báo hiệu" qua cờ chung.

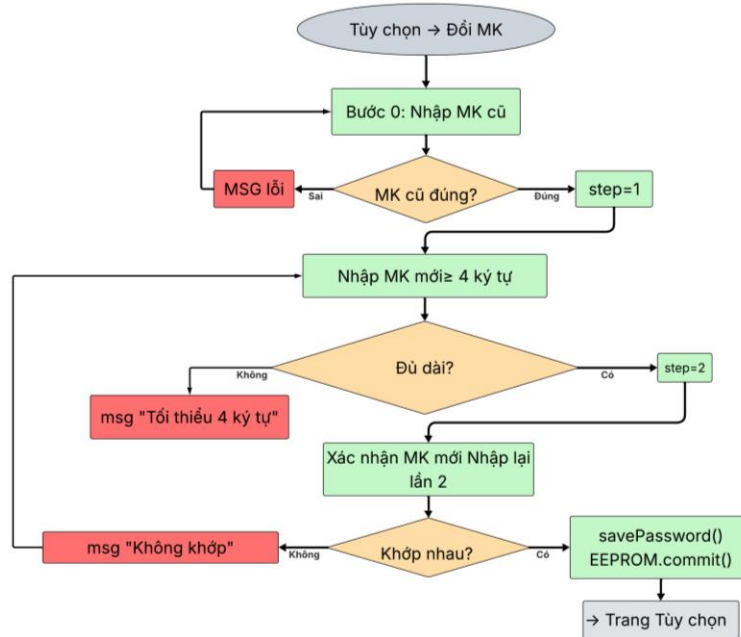
*Khối "vTaskDelay(2000ms)" (xám):*

Dù đi theo nhánh nào ở trên (trừ hai nhánh rẽ nhánh sớm quay thẳng về đầu), cuối cùng thiết bị đều dừng lại nghỉ đúng 2 giây bằng vTaskDelay. Đây là lệnh "nhường CPU" đúng cách trong FreeRTOS - khác với delay() thông thường, nó không chiếm giữ lõi CPU trong lúc chờ, giúp các tác vụ khác trên cùng lõi (nếu có) vẫn chạy được bình thường.

*Quay lại "Bắt đầu":*

Sau 2 giây nghỉ, vòng lặp while(true) quay lại đầu, lặp lại toàn bộ quy trình hỏi Firebase - cứ như vậy mãi mãi trong suốt thời gian thiết bị hoạt động, độc lập hoàn toàn với những gì đang diễn ra trên màn hình cảm ứng ở Core 1.

### 3.3.6. Luồng đổi mật khẩu



Hình 19 : Lưu đồ thuật toán đổi mật khẩu

#### Mô tả lưu đồ đổi mật khẩu:

Bắt đầu "Tùy chọn → Đổi MK":

Từ màn hình "TÙY CHỈNH", người dùng chạm vào nút "DOI MAT KHAU", thiết bị gọi drawPage(PAGE\_CHANGE\_PW). Hàm này đặt lại changePwStep = 0 (bước đầu tiên trong 3 bước: nhập MK cũ → nhập MK mới → xác nhận MK mới), rồi vẽ trang cùng bàn phím số 4x4 quen thuộc (giống trang mật khẩu), kèm 3 chấm tròn nhỏ phía trên ô nhập để thể hiện đang ở bước nào trong quy trình 3 bước.

Khởi "Bước 0: Nhập MK cũ" (xanh lá nhạt):

Ô nhập hiển thị nhãn "NHAP MAT KHAU CU" màu vàng. Người dùng gõ số bằng bàn phím (thêm ký tự vào inputStr, tối đa 8 ký tự, có thể DEL/CLR như các trang khác) rồi bấm OK để xác nhận, gọi hàm processChangePw().

Kiểm tra "MK cũ đúng?" (hình thoi):

So sánh inputStr vừa gõ với savedPassword hiện tại (đang lưu trong bộ nhớ):

Sai → đi sang khối đỏ "MSG lỗi": hiển thị thông báo "MAT KHAU CU SAI!" màu đỏ trong 1.8 giây, xóa inputStr, rồi quay lại chính Bước 0 để người dùng nhập lại mật khẩu cũ từ đầu chưa được đi tiếp.

Đúng → đi sang khối xanh step = 1: chuyển changePwStep sang bước 1, xóa inputStr và thông báo cũ, vẽ lại màn hình với nhãn mới. Chấm tròn thứ 2 sáng lên báo hiệu đã qua bước 1.

*Khối "Nhập MK mới  $\geq 4$  ký tự" (xanh lá nhạt):*

Ô nhập đổi nhãn thành "NHAP MAT KHAU MOI" màu xanh dương. Người dùng gõ mật khẩu mới mong muốn bằng bàn phím, rồi bấm OK.

*Kiểm tra "Đủ dài?" (hình thoi):*

Kiểm tra độ dài chuỗi vừa gõ có từ 4 ký tự trở lên hay không (giới hạn tối thiểu để tránh mật khẩu quá yếu, ví dụ chỉ 1-2 số):

Không → đi sang khối đỏ "msg Tối thiểu 4 ký tự": hiện cảnh báo màu cam/đỏ "Toi thieu 4 ky tu!" trong 1.8 giây, xóa inputStr, và quay lại chính bước nhập MK mới này (đường mũi tên vòng trái quay lên ô "Nhập MK mới") để người dùng gõ lại mật khẩu cũ vẫn giữ nguyên, chưa đổi gì.

Có → đi sang khối xanh step = 2: lưu tạm chuỗi vừa gõ vào biến newPassStr (mật khẩu mới dự kiến, chưa ghi xuống EEPROM), chuyển changePwStep sang bước 2, xóa inputStr để chuẩn bị cho lượt gõ xác nhận. Chấm tròn thứ 3 sáng lên.

*Khối "Xác nhận MK mới. Nhập lại lần 2" (xanh lá nhạt):*

Ô nhập đổi nhãn thành "XAC NHAN MAT KHAU MOI" màu cam, yêu cầu người dùng gõ lại đúng y hệt mật khẩu mới vừa đặt ở bước trên, để chắc chắn không bị gõ nhầm/gõ lỗi tay. Người dùng gõ xong rồi bấm OK.

*Kiểm tra "Khớp nhau?" (hình thoi):*

So sánh inputStr (lần gõ thứ 2) với newPassStr (lần gõ thứ 1 đã lưu tạm):

Không → đi sang khối đỏ "msg Không khớp": hiện thông báo "KHONG KHOP! Nhap lai" màu đỏ, xóa cả inputStr lẫn newPassStr, đưa changePwStep quay về bước 1 (đường mũi tên nét đứt vòng ngược lại tận ô "Nhập MK mới  $\geq 4$  ký tự" ở bên trái) nghĩa là người dùng



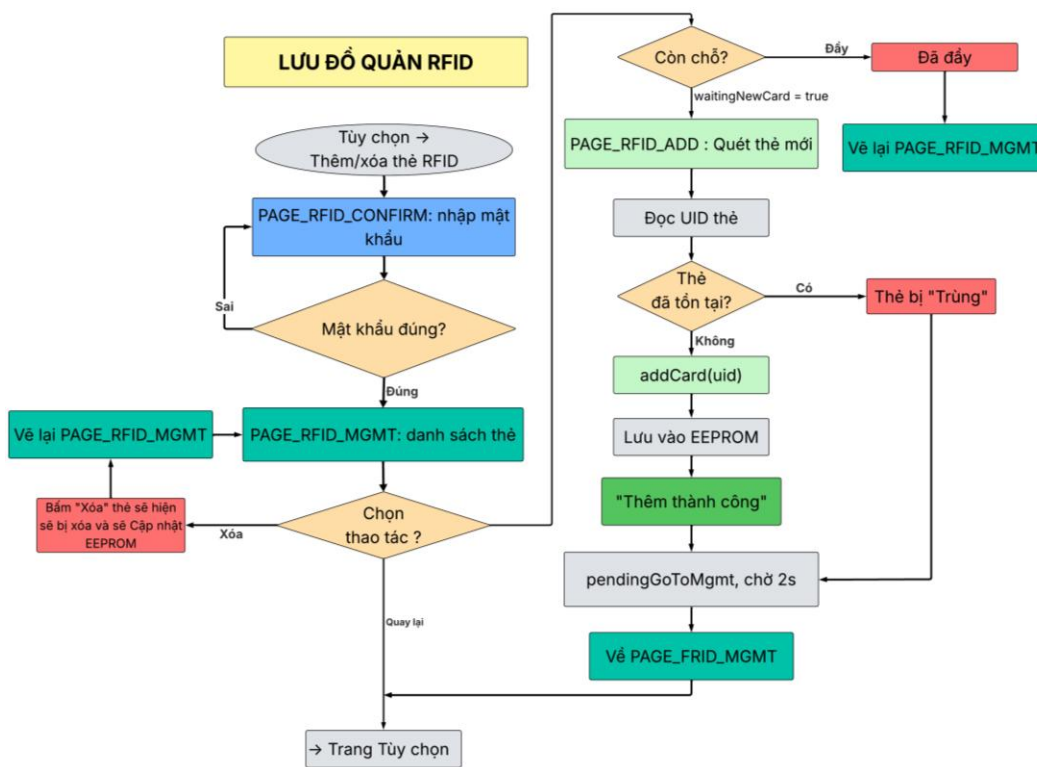
phải gõ lại mật khẩu mới từ đầu (không phải chỉ gõ lại bước xác nhận), mật khẩu cũ trong bộ nhớ vẫn chưa hề bị thay đổi.

Có → đi sang khối xanh dương "savePassword(); EEPROM.commit()": ghi inputStr (mật khẩu mới đã xác nhận khớp) vào biến savedPassword trong bộ nhớ RAM, sau đó gọi savePassword() để ghi từng byte xuống EEPROM và gọi EEPROM.commit() để lưu thật sự xuống bộ nhớ flash đảm bảo mật khẩu mới còn tồn tại kể cả khi mất điện. Hiện thông báo "DOI MAT KHAU OK!" màu xanh lá trong 1.5 giây.

Khởi "→ Trang Tùy chọn" (kết thúc):

Sau khi đổi mật khẩu thành công và hiển thị thông báo xong, thiết bị tự động gọi drawPage(PAGE\_TUYCHON), quay về màn hình "TÙY CHỈNH" ban đầu kết thúc toàn bộ quy trình đổi mật khẩu 3 bước. Lưu ý: nút << (quay lại) trên bàn phím cũng có thể được bấm ở bất kỳ bước nào trong 3 bước trên để hủy ngang quy trình, quay thẳng về trang TUY CHINH mà không lưu bất kỳ thay đổi nào nhánh này không thể hiện trong ảnh nhưng có trong code.

### 3.3.7. Luồng thêm và xóa thẻ Rfid



Hình 20 : Lưu đồ thuật toán thêm hoặc xóa thẻ Rfid

### **Mô tả lưu đồ thêm/xóa thẻ Rfid:**

*Bắt đầu "Tùy chọn → Thêm/xóa thẻ RFID":*

Từ màn hình "TÙY CHỈNH", người dùng chạm vào nút "THEM/XOA RFID", thiết bị gọi `drawPage(PAGE_RFID_CONFIRM)` chuyển sang trang xác nhận quyền trước khi cho phép chỉnh sửa danh sách thẻ, coi như một lớp bảo vệ bằng mật khẩu admin.

*Khối "PAGE\_RFID\_CONFIRM: nhập mật khẩu" (xanh dương)*

Trang này vẽ ô nhập với nhãn "NHAP MAT KHAU DE TIEP TUC" màu cam, cùng bàn phím số 4x4 y hệt các trang khác. Người dùng gõ mật khẩu rồi bấm OK, gọi hàm `checkRFIDConfirmPassword()`.

*Kiểm tra "Mật khẩu đúng?" (hình thoi):*

So sánh chuỗi vừa gõ với `savedPassword` đang lưu (cùng mật khẩu dùng để mở cửa, không phải mật khẩu riêng cho RFID):

Sai → hiện thông báo "SAI MAT KHAU!" màu đỏ trong 1.8 giây, xóa `inputStr`, và quay lại chính ô nhập mật khẩu (mũi tên vòng trái lên khối "PAGE\_RFID\_CONFIRM") để người dùng thử lại chưa được vào trang quản lý thẻ.

Đúng → xóa `inputStr`, chuyển sang `drawPage(PAGE_RFID_MGMT)` vào thẳng danh sách thẻ.

*Khối "PAGE\_RFID\_MGMT: danh sách thẻ" (xanh lá đậm):*

Trang này đặt `selectedCard = -1` (chưa chọn thẻ nào), rồi vẽ toàn bộ danh sách thẻ đã đăng ký (tối đa 5 thẻ, hiển thị dạng [1] UID..., [2] UID...), mỗi thẻ có kèm một nút nhỏ "XOA" bên phải. Phía dưới danh sách là nút lớn "+ THEM THE MOI" (bị làm mờ nếu đã đủ 5 thẻ) và nút "QUAY LAI", cùng dòng chữ hiển thị số thẻ hiện có (ví dụ "3/5 the").

*Kiểm tra "Chọn thao tác?" (hình thoi)*

Thiết bị chờ người dùng chạm vào một trong các vùng đã vẽ, có 3 khả năng:

Xóa → người dùng bấm nút "XOA" cạnh một thẻ cụ thể trong danh sách. Đi sang khối đổ "Bấm Xóa thẻ sẽ hiện sẽ bị xóa và sẽ Cập nhật EEPROM": gọi `deleteCard(idx)` dời các thẻ phía sau lên một vị trí trong mảng, giảm `cardCount` đi 1, ghi lại toàn bộ danh sách xuống EEPROM (`saveCards()`) để không mất khi cúp điện. Hiện thông báo "DA XOA THE!" màu xanh lá, đặt `selectedCard = -1`, rồi vẽ lại chính trang `PAGE_RFID_MGMT`

(mũi tên vòng trái "Về lại PAGE\_RFID\_MGMT") để cập nhật danh sách mới ngay tại chỗ không rời khỏi trang quản lý.

Quay lại → bấm nút "QUAY LAI", thoát thẳng về "Trang Tùy chọn" (drawPage(PAGE\_TUYCHON)), kết thúc quy trình quản lý thẻ.

Thêm thẻ mới (nhánh dẫn sang cụm bên phải) → nếu bấm vào một dòng thẻ (không phải nút Xóa) thì chỉ đơn giản chọn/bỏ chọn dòng đó để tô sáng (không có tác động gì khác); còn nếu bấm nút "+ THEM THE MOI" thì rẽ sang cụm xử lý thêm thẻ ở bên phải sơ đồ.

*Kiểm tra "Còn chỗ?" (hình thoi, cụm bên phải):*

Trước khi cho thêm thẻ, thiết bị kiểm tra cardCount < MAX\_CARDS (đã có dưới 5 thẻ hay chưa):

Đầy → đi sang khối đỏ "Đã đầy": hiện thông báo "DA DAY (toi da 5 the)!" màu đỏ ngay tại trang quản lý, rồi khối "Về lại PAGE\_RFID\_MGMT" vẽ lại trang hiện tại không cho phép sang trang thêm thẻ.

waitingNewCard = true → đủ chỗ, thiết bị đặt cờ waitingNewCard = true rồi gọi drawPage(PAGE\_RFID\_ADD), chuyển hẳn sang trang thêm thẻ mới.

*Khối "PAGE\_RFID\_ADD: Quét thẻ mới" (xanh lá nhạt):*

Trang này vẽ một khung lớn với biểu tượng "[ THE RFID MOI ]" màu xanh lá, dòng chữ nhắc "Dua the can them vao dau doc...", số lượng thẻ hiện có ("Dang co: x/5 the"), và một nút "HUY" phía dưới để hủy ngang thao tác thêm thẻ, quay thẳng về danh sách quản lý mà không thêm gì. Trong lúc chờ, hàm quét RFID nền (scanRFID(), chạy mỗi 300ms trong loop() chính) vẫn hoạt động song song - khi phát hiện thẻ, vì đang ở đúng PAGE\_RFID\_ADD nên nó gọi handleRFIDAddScan() thay vì xử lý mở cửa như bình thường.

*Khối "Đọc UID thẻ" (xám):*

Thiết bị đọc mã UID của thẻ vừa quét được (cùng cơ chế đọc như lúc quét mở cửa), lưu vào rfidUID.

*Kiểm tra "Thẻ đã tồn tại?" (hình thoi):*

So sánh UID vừa quét với từng thẻ đã có sẵn trong cardList:

Có (trùng) → đi sang khối đỏ "Thẻ bị Trùng": hiện thông báo "THE DA TON TAI!" màu cam/đỏ. Thẻ này không được thêm lần nữa (tránh trùng lặp vô nghĩa trong danh sách).

Không (chưa có) → đi sang khối xanh addCard(uid): gọi hàm thêm thẻ.

*Khối "addCard(uid)" (xanh lá nhạt):*

Hàm này kiểm tra lại một lần nữa còn chỗ trống không (cardCount < MAX\_CARDS) và không trùng UID (kiểm tra kép cho chắc), rồi ghi UID vào vị trí trống tiếp theo trong mảng cardList, tăng cardCount lên 1.

*Khối "Lưu vào EEPROM" (xám):*

Gọi saveCards(): ghi lại byte đếm số thẻ mới và toàn bộ mảng UID (kể cả các ô trống được ghi đè bằng 0) xuống EEPROM, rồi EEPROM.commit() để đảm bảo dữ liệu tồn tại lâu dài kể cả khi mất điện.

*Khối "Thêm thành công" (xanh lá đậm):*

Hiện thông báo "THEM THE THANH CONG!" màu xanh lá ngay trên trang PAGE\_RFID\_ADD, đồng thời đặt waitingNewCard = false (không chờ quét thêm thẻ nào nữa trong phiên này).

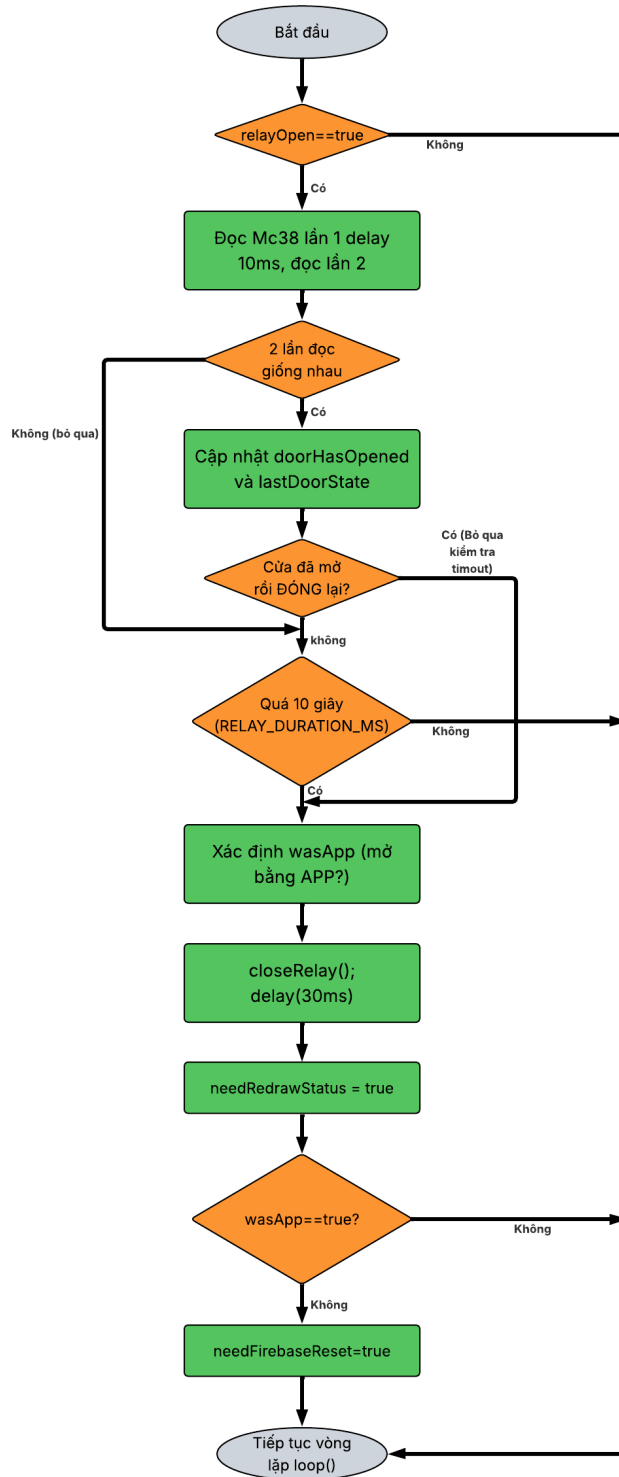
*Khối "pendingGoToMgmt, chờ 2s" (xám):*

Thiết bị đặt cờ pendingGoToMgmt = true và ghi lại thời điểm hiện tại (addCardTimer). Ở đầu vòng lặp loop() chính có đoạn kiểm tra riêng: hễ cờ này bật và đã trôi qua hơn 2 giây thì tự động chuyển trang đây là khoảng nghỉ để người dùng kịp đọc thông báo "THEM THANH CONG!" trước khi màn hình đổi tiếp, đồng thời trong lúc chờ này, hàm quét RFID nền tạm ngưng hoạt động để tránh quét chồng thẻ cũ.

*Khối "Về PAGE\_RFID\_MGMT" (xanh lá đậm)*

Sau 2 giây, cờ pendingGoToMgmt được tắt, thiết bị gọi drawPage(PAGE\_RFID\_MGMT), quay trở lại đúng danh sách quản lý thẻ giờ đã hiển thị thêm thẻ vừa được thêm vào, khép kín vòng quay lại đầu cụm bên phải (mũi tên nét đứt vòng lên trên tới khối "Còn chỗ?"/danh sách chính).

### **3.3.8. Luồng xử lý relay và MC-38**



Hình 21 : Lưu đồ thuật toán xử lý relay và cảm biến từ MC-38

**Mô tả lưu đồ xử lý relay và MC-28:**

*Bắt đầu*

Đây là đoạn code nằm ngay trong loop() chính (Core 1), được kiểm tra ở mỗi vòng lặp, chạy hàng ngàn lần mỗi giây, chuyên trách việc theo dõi cửa để tự động khóa lại đúng lúc.

*Kiểm tra "relayOpen == true" (hình thoi):*

Thiết bị hỏi biến toàn cục relayOpen xem cửa hiện có đang mở (relay đang ON) hay không: Không → nghĩa là cửa đang khóa sẵn, không có gì để theo dõi cả, bỏ qua toàn bộ khối xử lý bên dưới, đi thẳng luôn tới "Tiếp tục vòng lặp loop()" (mũi tên nét đậm bên phải chạy dọc xuống cuối).

Có → cửa đang mở, cần theo dõi xem người dùng đã đóng cửa lại chưa, đi tiếp xuống bước đọc cảm biến.

*Khối "Đọc MC38 lần 1, delay 10ms, đọc lần 2" (xanh lá):*

Thiết bị đọc trạng thái chân MC38\_PIN (cảm biến từ gắn trên cửa) lần thứ nhất, sau đó delay(10) (chờ 10 mili-giây), rồi đọc lại lần thứ hai. Đây là kỹ thuật lọc nhiễu (debounce) đơn giản vì công tắc từ cơ khí khi đóng/mở có thể rung nhẹ trong vài mili-giây gây tín hiệu nhiễu giả, đọc 2 lần cách nhau một chút giúp chắc chắn hơn tín hiệu là thật.

*Kiểm tra "2 lần đọc giống nhau" (hình thoi):*

So sánh kết quả đọc lần 1 và lần 2:

Không (hai lần đọc khác nhau đúng lúc bị nhiễu hoặc đúng lúc cửa đang dao động) → nhãn "Không (bỏ qua)": thiết bị không tin kết quả này, bỏ qua toàn bộ phần xử lý trạng thái cửa ở vòng này, nhảy thẳng xuống thẳng bước kiểm tra "Cửa đã mở rồi ĐÓNG lại?" mà không cập nhật lastDoorState để lần đọc tiếp theo (vòng loop() kế tiếp, gần như ngay lập tức) thử lại từ đầu.

Có (hai lần đọc giống nhau tín hiệu ổn định, đáng tin) → đi tiếp xuống bước cập nhật trạng thái.

*Khối "Cập nhật doorHasOpened và lastDoorState" (xanh lá):*

Thiết bị kiểm tra: nếu trước đó cửa chưa từng ghi nhận là đã mở (!doorHasOpened) và trạng thái cũ là LOW còn trạng thái mới đọc là HIGH, thì đánh dấu doorHasOpened = true (tức "đã ghi nhận cửa có mở ra thật, ít nhất một lần, kể từ lúc relay bật") và in log "Cua DA MO" ra Serial. Sau đó lastDoorState luôn được cập nhật bằng giá trị vừa đọc để dùng làm mốc so sánh cho vòng lặp kế tiếp.

*Kiểm tra "Cửa đã mở rồi ĐÓNG lại?" (hình thoi):*

Đây là điều kiện quan trọng nhất: thiết bị chỉ coi là "cửa vừa đóng lại" khi đã từng ghi nhận mở ra trước đó (`doorHasOpened == true`) và trạng thái vừa chuyển từ HIGH (mở) xuống LOW (đóng) tránh trường hợp bị khóa nhầm ngay khi vừa mở relay mà cửa chưa kịp mở ra (`lastDoorState` ban đầu là LOW nên nếu không kiểm tra `doorHasOpened` thì sẽ tưởng cửa "đóng" ngay từ đầu):

Không → chưa đủ điều kiện xem là cửa vừa đóng lại (có thể cửa vẫn đang mở, hoặc chưa từng mở ra lần nào), đi tiếp xuống bước kiểm tra timeout 10 giây.

Có (bỏ qua kiểm tra timeout) → đúng là cửa vừa đóng lại sau khi đã mở - nhánh này rẽ thẳng (mũi tên cong lên trên bên phải) bỏ qua hẳn bước kiểm tra timeout 10 giây, đi thẳng xuống khối "Xác định wasApp" để khóa cửa ngay lập tức - vì cửa đã đóng thật rồi thì không cần đợi hết 10 giây mới khóa nữa, khóa ngay cho an toàn.

*Kiểm tra "Quá 10 giây (RELAY\_DURATION\_MS)" (hình thoi):*

Nếu chưa phát hiện được cửa đóng ở bước trên, thiết bị kiểm tra thêm: đã trôi qua hơn 10 giây (`RELAY_DURATION_MS`) kể từ lúc `relayTimer` được đặt (tức từ lúc bắt đầu mở cửa) hay chưa:

Không → chưa quá 10 giây và cửa cũng chưa đóng, thiết bị chưa làm gì cả, thoát khỏi toàn bộ nhánh xử lý cửa của vòng lặp này (mũi tên bên phải chạy thẳng xuống "Tiếp tục vòng lặp `loop()`") cửa vẫn tiếp tục mở, đợi vòng `loop()` kế tiếp kiểm tra lại.

Có → đã quá 10 giây mà cửa vẫn chưa được ai đóng lại (hoặc cảm biến không phát hiện được), thiết bị coi đây là tình huống hết giờ, in log "Timeout 10s -> KHOA" ra Serial, rồi cũng đi xuống khối "Xác định wasApp" để khóa cửa cưỡng bức đảm bảo cửa không bị mở quá lâu dù người dùng quên đóng hay cảm biến bị lỗi.

*Khối "Xác định wasApp (mở bằng APP?)" (xanh lá):*

Trước khi khóa cửa, thiết bị kiểm tra xem lần mở cửa này có phải được kích hoạt từ App/Firebase hay không, bằng cách xem `openSource == "APP"` hoặc biến `relay_firebase` đang bật lưu kết quả vào biến tạm `wasApp`. Việc này cần làm trước khi gọi khóa cửa, vì hàm khóa cửa sẽ xóa sạch `openSource` ngay sau đó.

*Khối "closeRelay(); delay(30ms)" (xanh lá):*

Gọi `closeRelay()`: hạ chân relay xuống mức OFF để khóa cửa thật sự bằng phần cứng, đặt lại `relayOpen = false`, xóa `openSource` về rỗng, và reset `doorHasOpened` về false (chuẩn bị sẵn cho lần mở cửa kế tiếp). Sau đó `delay(30)` khoảng nghỉ ngắn 30 mili-giây để module RFID (dùng chung bus SPI) kịp "nhả" đường truyền trước khi vẽ lại màn hình, tránh xung đột bus SPI giữa việc vẽ TFT và việc RFID đang giao tiếp.

*Khởi "needRedrawStatus = true" (xanh lá):*

Đặt cờ này lên true để báo cho phần code vẽ giao diện (chạy ngay bên dưới trong cùng vòng `loop()`) biết cần vẽ lại thanh trạng thái/footer, cập nhật hiển thị từ "CUA DANG MO" sang "DONG CUA".

*Kiểm tra "wasApp == true?" (hình thoi):*

Đây là bước quyết định có cần báo ngược lại cho Firebase hay không:

Không (cửa vừa đóng không phải do mở từ App) → không cần làm gì thêm với Firebase, đi thẳng luôn (mũi tên bên phải) tới "Tiếp tục vòng lặp `loop()`".

(đúng - cửa vừa mở là do App) → đi tiếp xuống khối cuối để reset lại Firebase.

*Khởi "needFirebaseReset = true" (xanh lá):*

Chỉ khi cửa vừa mở bằng App thì thiết bị mới đặt cờ này lên true. Cờ này được một đoạn khác ở đầu `loop()` đọc và xử lý: gửi HTTP PUT ghi giá trị `RELAY_KEY` về lại "0" trên Firebase - để app điện thoại biết cửa đã tự đóng lại, tránh trường hợp giá trị "1" cũ còn treo mãi trên Firebase khiến app tưởng cửa vẫn đang mở, hoặc khiến thiết bị hiểu nhầm lệnh mở cửa lặp lại ở vòng kiểm tra Firebase kế tiếp.

*Khởi "Tiếp tục vòng lặp `loop()`" (kết thúc):*

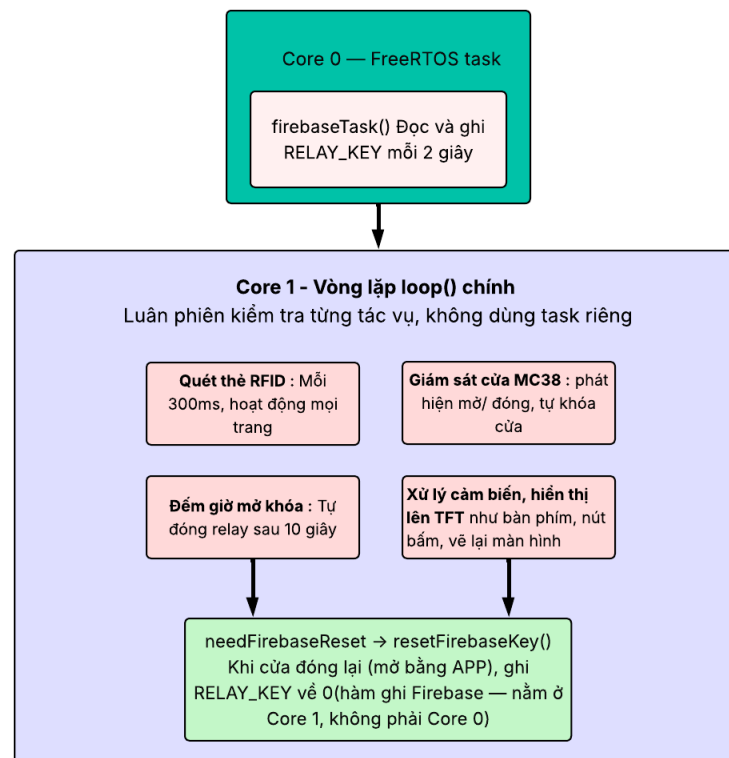
Toàn bộ nhánh xử lý cửa/relay của vòng `loop()` hiện tại kết thúc tại đây chương trình đi tiếp sang các đoạn xử lý khác trong cùng `loop()` (cập nhật footer, quét RFID, xử lý cảm ứng...), rồi quay lại đầu `loop()` gần như ngay lập tức để kiểm tra lại từ đầu ("`relayOpen == true?`"), lặp vô hạn suốt vòng đời hoạt động của thiết bị.

### **3.3.9. Các luồng hoạt động các task nền song song**

ESP32 có 2 nhân xử lý (dual-core), và hệ thống tận dụng điều này để tách biệt tác vụ mạng khỏi tác vụ giao diện. Nhân Core 0 được giao riêng cho task `firebaseTask()`, chuyên trách đọc/ghi giá trị `RELAY_KEY` với Firebase mỗi 2 giây vì đây là tác vụ có độ trễ



mạng không ổn định, nếu để chung với vòng lặp chính sẽ làm màn hình và cảm ứng bị "đứng hình" khi mạng chậm. Trong khi đó, Core 1 đảm nhiệm toàn bộ vòng lặp loop() chính, luân phiên kiểm tra tuần tự các tác vụ còn lại: quét thẻ RFID, giám sát cảm biến cửa MC38, đếm giờ tự khóa, và xử lý cảm ứng/hiển thị. Sơ đồ dưới đây minh họa sự phân chia này.



Hình 22 : Luồng hoạt động các task nền song song

### Mô tả luồng hoạt động của các task:

Core 0 — Task Firebase (chạy nền, độc lập)

Một tác vụ riêng biệt, chỉ làm một việc: cứ mỗi 2 giây thì đọc và ghi giá trị RELAY\_KEY trên Firebase một lần, để biết app có ra lệnh mở cửa hay không.

Core 1 — Vòng lặp loop() chính (tuần tự)

Đây là nơi xử lý mọi thứ còn lại của hệ thống, luân phiên kiểm tra từng việc một chứ không tách task riêng:

Quét thẻ RFID — mỗi 300ms, hoạt động ở mọi trang

Giám sát cửa (cảm biến MC38) — phát hiện cửa mở/đóng, tự khóa lại khi cửa đóng

Đếm giờ mở khóa — tự động đóng relay sau 10 giây nếu chưa có ai đóng cửa

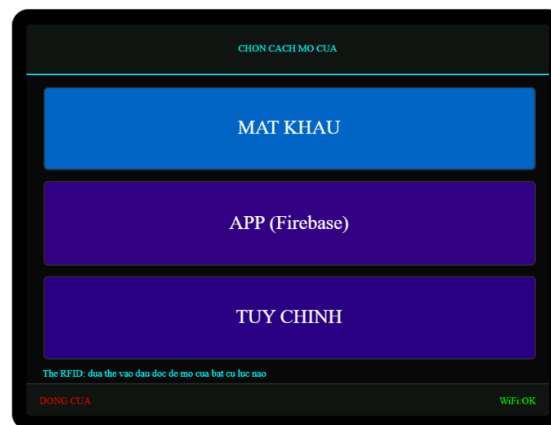
Xử lý cảm biến, hiển thị lên TFT — như bàn phím, nút bấm, vẽ lại màn hình

Điểm nối giữa 2 luồng

Cả 2 nhánh "Đếm giờ mở khóa" và "Giám sát cửa MC38" đều có thể dẫn tới bước cuối: khi cửa đóng lại (trong trường hợp mở bằng APP), hệ thống gọi `resetFirebaseKey()` để ghi `RELAY_KEY` về 0 — và thao tác ghi này nằm ở Core 1, không phải Core 0, dù Core 0 là nơi đọc dữ liệu từ Firebase.

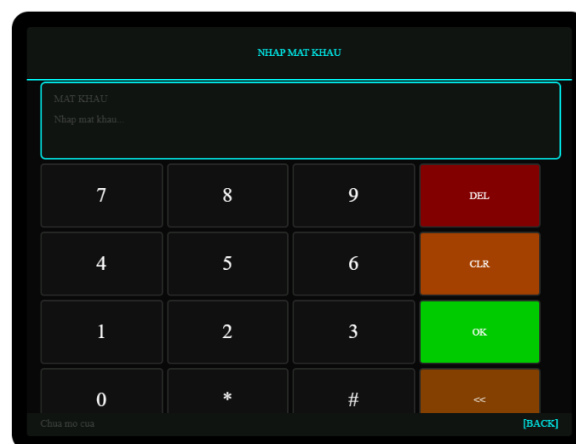
### 3.4. Giao diện màn hình cảm ứng

Giao diện trang chủ mặc định:



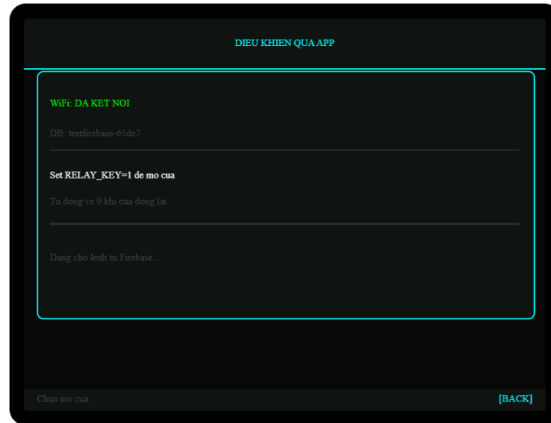
Hình 23 : Giao diện trang chủ

Các giao diện khi chọn “MẬT KHẨU”



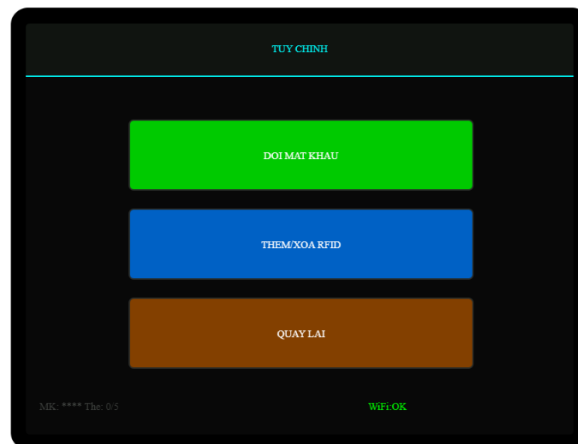
Hình 24 : Giao diện trang nhập mật khẩu

Các giao diện khi chọn mở bằng “App (Firebase)”



Hình 25 : Giao diện trang mở cửa bằng APP

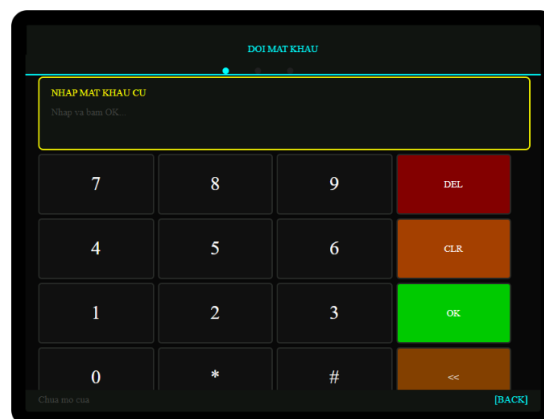
### Các giao diện khi chọn chế độ “TÙY CHỈNH”



Hình 26 : Giao diện trang tùy chọn

### Khi chọn đổi “MẬT KHẨU”

Giao diện xác nhận mật khẩu cũ:



Hình 27 : Giao diện trang xác nhận đổi mật khẩu

Giao diện nhập mật khẩu mới:

ĐỔI MẬT KHẨU

NHẬP MẬT KHẨU MỚI

Nhập và bấm OK...

7 8 9 DEL

4 5 6 CLR

1 2 3 OK

0 \* # <<

Chưa tạo tài [BACK]

Hình 28 : Giao diện trang nhập mật khẩu mới

### Khi chọn “THÊM/XÓA RFID”

Xác nhận bằng mật khẩu:

XÁC NHẬN QUYỀN TRUY CẬP

NHẬP MẬT KHẨU ĐỂ TIẾP TỤC

Nhập mật khẩu admin...

7 8 9 DEL

4 5 6 CLR

1 2 3 OK

0 \* # <<

Chưa tạo tài [BACK]

Hình 29 : Giao diện trang xác nhận thêm/xóa thẻ Rfid

Giao diện quản lý thẻ:

QUẢN LÝ THE RFID

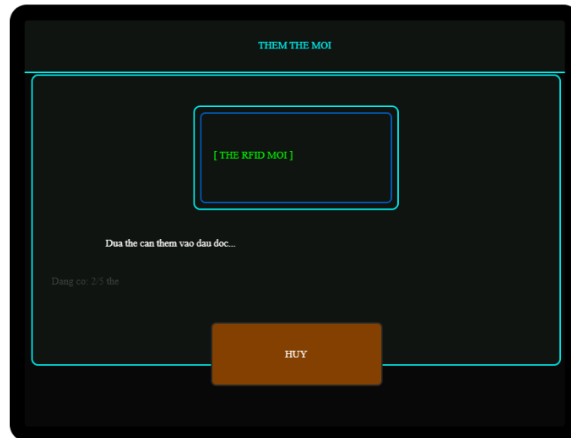
[1] 04 A2 3F 1C XOA

[2] 12 BB C4 09 XOA

+ THÊM THE MỚI 2/5 the QUAY LẠI

*Hình 30 : Giao diện trang quản lý thẻ Rfid*

Giao diện thêm thẻ:



*Hình 31 : Giao diện thêm thẻ Rfid*

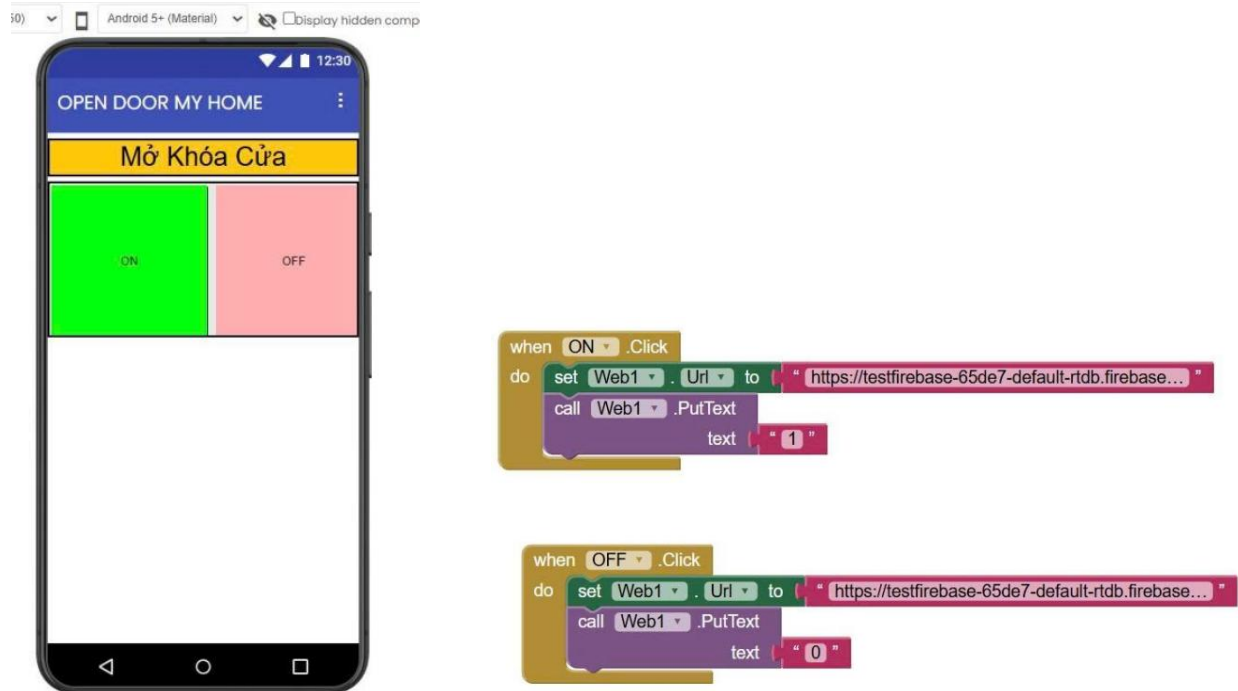
### 3.5. Giao diện ứng dụng App Inventor

Ứng dụng được xây dựng bằng nền tảng MIT App Inventor với tên "OPEN DOOR MY HOME". Giao diện đơn giản, trực quan gồm một tiêu đề "Mở Khóa Cửa" và hai nút bấm: nút ON màu xanh lá và nút OFF màu hồng đặt cạnh nhau, giúp người dùng dễ dàng thao tác chỉ với một chạm.

Về mặt lập trình khởi lệnh, ứng dụng hoạt động theo cơ chế sau:

Khi người dùng nhấn nút ON, ứng dụng gọi thành phần Web1 để thực hiện lệnh HTTP PUT ghi giá trị "1" lên node RELAY\_KEY trên Firebase. ESP32 phát hiện sự thay đổi này và kích hoạt relay mở khóa.

Khi người dùng nhấn nút OFF, ứng dụng tương tự ghi giá trị "0" lên Firebase, báo hiệu đóng khóa. Tuy nhiên trong thực tế, hệ thống cũng tự động reset về 0 khi cảm biến MC38 phát hiện cửa đã đóng, nên người dùng không nhất thiết phải bấm OFF thủ công.



Hình 32 : Giao diện ứng dụng App Inventor

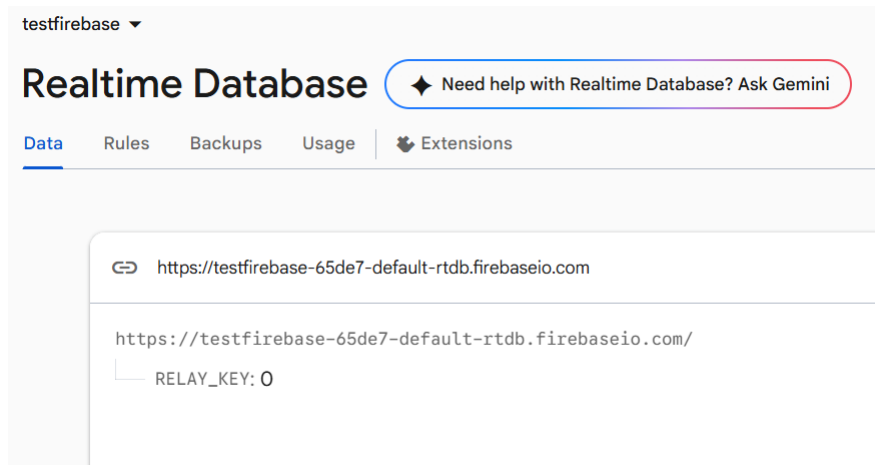
### 3.6. Thực hiện Firebase

Cơ sở dữ liệu thời gian thực (Firebase Realtime Database)

Hệ thống sử dụng Firebase Realtime Database của Google làm trung gian truyền lệnh điều khiển từ xa giữa ứng dụng di động và thiết bị ESP32. Database được đặt tại máy chủ khu vực us-central1 với tên project là testfirebase-65de7.

Cấu trúc dữ liệu được thiết kế đơn giản, chỉ gồm một node duy nhất là RELAY\_KEY lưu trữ giá trị số nguyên. Khi người dùng muốn mở khóa qua ứng dụng, ứng dụng sẽ ghi giá trị 1 vào node này. ESP32 liên tục truy vấn Firebase mỗi 2 giây thông qua HTTP GET, khi phát hiện giá trị thay đổi thành 1 thì kích hoạt relay mở khóa trong 10 giây. Sau khi cửa đóng lại (cảm biến MC38 phát hiện), hệ thống tự động ghi giá trị 0 trở lại Firebase để reset trạng thái, sẵn sàng nhận lệnh tiếp theo.

Ưu điểm của cách tiếp cận này là không cần thiết lập máy chủ riêng, hoạt động ổn định miễn là có kết nối Internet, và dữ liệu đồng bộ gần như tức thời nhờ cơ chế polling 2 giây.



*Hình 33 : Trang Firebase Realtime Database*

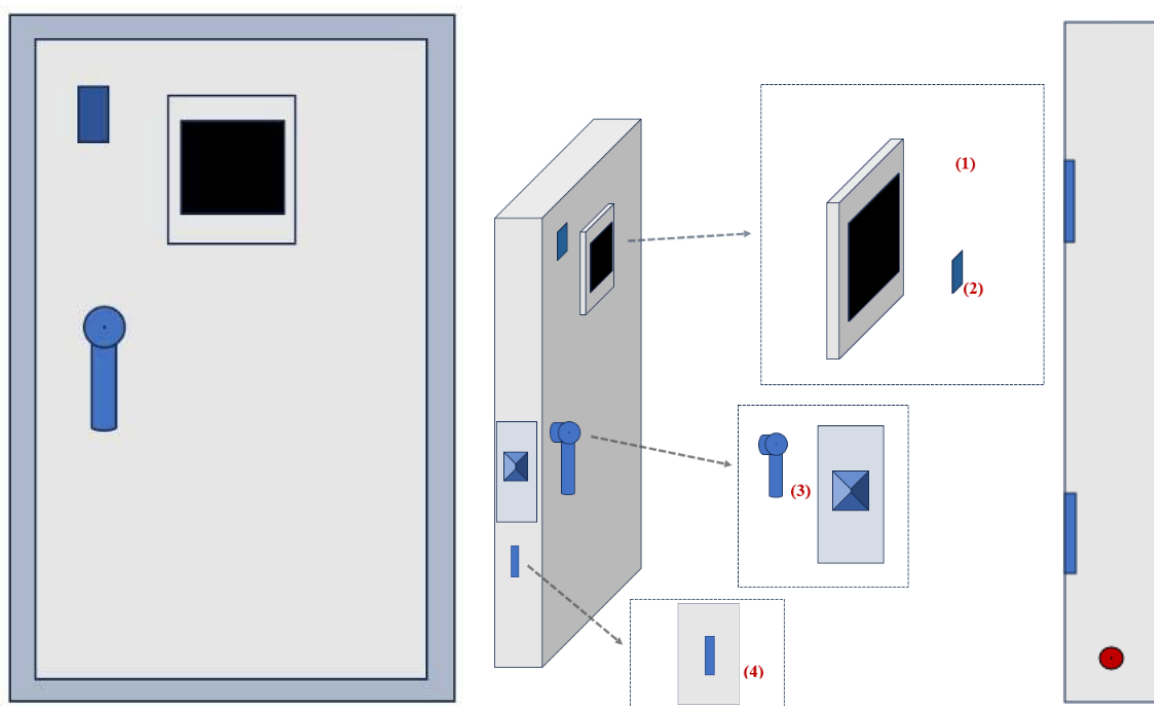
Hình trên là giao diện Firebase Realtime Database của dự án testfirebase-65de7. Cơ sở dữ liệu lưu trữ một trường duy nhất là RELAY\_KEY với giá trị mặc định là 0.

Khi người dùng muốn mở khóa từ xa, chỉ cần thay đổi giá trị RELAY\_KEY từ 0 lên 1 thông qua app hoặc trực tiếp trên console. ESP32 sẽ phát hiện sự thay đổi này trong vòng 2 giây thông qua HTTP GET và kích hoạt relay mở khóa. Sau khi cửa đóng lại, cảm biến MC-38 báo tín hiệu về ESP32 và hệ thống tự động gửi lệnh PUT để reset RELAY\_KEY về 0, sẵn sàng cho lần mở tiếp theo.

### **3.7. Mô hình cửa thiết kế**

Mô hình cửa thực tế được thiết kế nhằm minh họa bố trí các thiết bị phần cứng trên cửa trong hệ thống khóa cửa thông minh. Mô hình gồm một cánh cửa chính và khung cửa, trên đó tích hợp các thành phần điều khiển và cảm biến như sau:

- (1) Màn hình cảm ứng TFT ILI9341 hiển thị giao diện người dùng (bàn phím nhập mật khẩu, trạng thái hệ thống, menu tùy chọn).
- (2) Đầu đọc thẻ RFID (module MFRC522) dùng để quét thẻ khi mở cửa bằng thẻ từ.
- (3) Tay nắm cửa tích hợp khóa điện (relay điều khiển chốt khóa) cơ cấu chấp hành thực hiện việc mở/ khóa cửa.
- (4) Cảm biến từ MC38 gắn ở mép cửa và khung cửa để phát hiện trạng thái đóng/mở của cửa, phục vụ logic tự động khóa lại sau khi cửa đóng.



Hình 34 : Ý tưởng mô hình cửa

Mô hình cửa thực tế hoàn thành:



Hình 35 : Mô hình cửa đã được thiết kế



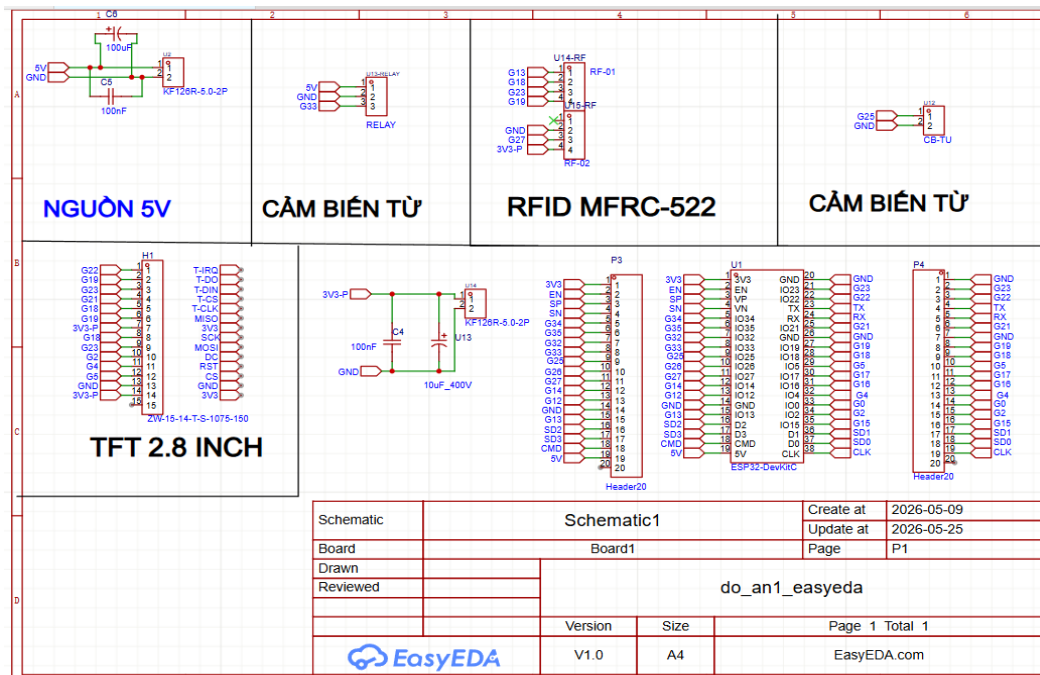
## PHẦN 2: KẾT QUẢ

### CHƯƠNG 4: KẾT QUẢ VÀ ĐÁNH GIÁ

#### 4.1. Thiết kế và thi công mạch PCB:

##### 4.1.1. Thiết kế Schematic các khối của mạch PCB

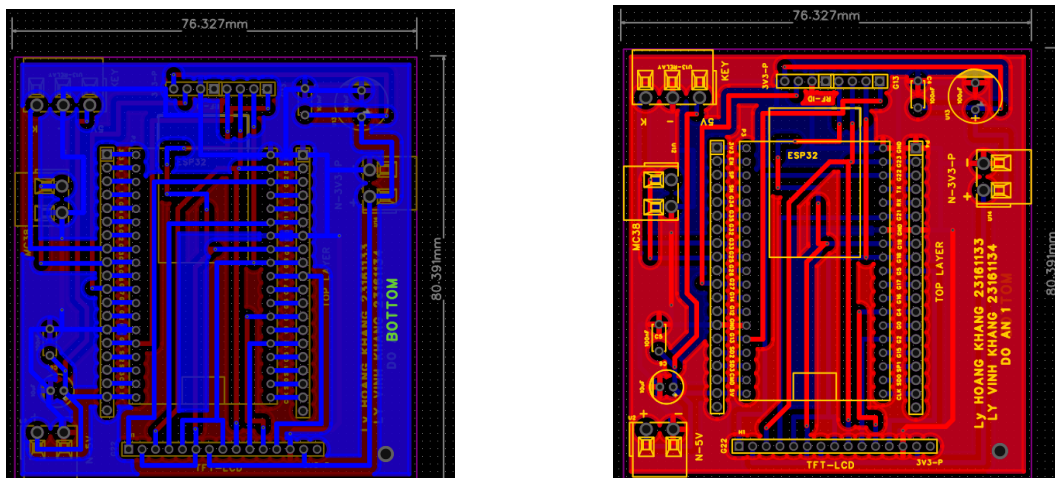
Sơ đồ mạch được thiết kế trên EasyEDA Pro :



Hình 36 : Schematic các khối

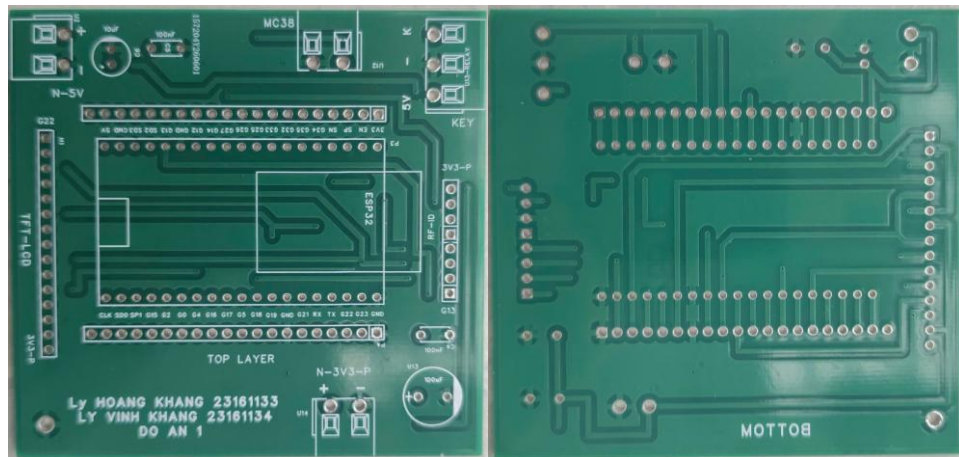
##### 4.1.2. Thiết kế mạch in PCB

Bottom và Top của PCB:



Hình 37 : PCB sau khi vẽ xong

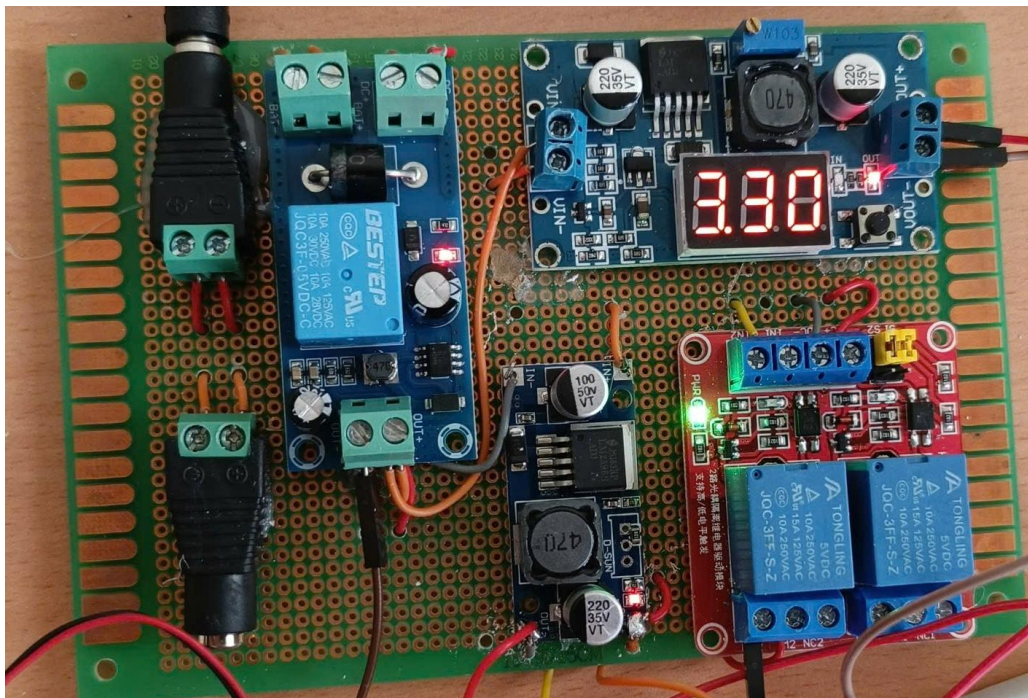
Mạch sau khi in:



Hình 38 : Mạch PCB sau khi in

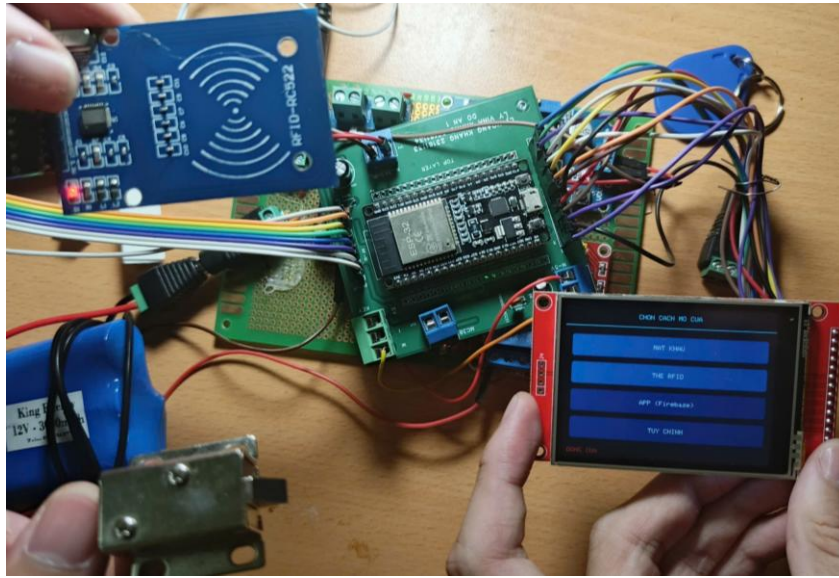
## 4.2. Thi công

Thi công khối nguồn:



Hình 39 : Khối cung cấp nguồn

*Khối trung tâm:*

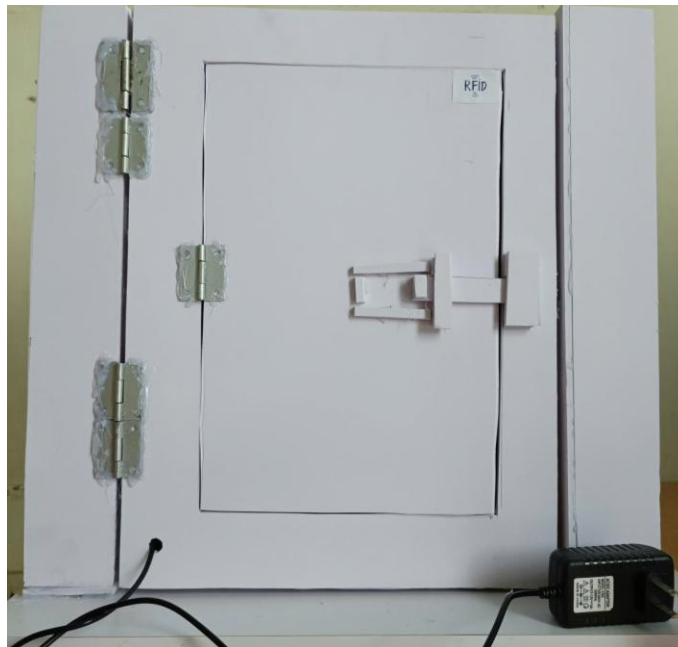


*Hình 40 : Mạch trung tâm sau khi hàn và lắp linh kiện*

#### **4.3. Mô hình cánh cửa thực tế sau khi hoàn thành:**

Mô hình cửa được nhóm thiết kế với kích thước 40cm × 30cm × 4cm, thu nhỏ theo tỷ lệ so với cửa thực tế nhưng vẫn đảm bảo đủ không gian để lắp đặt và trình diễn đầy đủ các chức năng của hệ thống kiểm soát ra vào.

*Phía trước cửa:*



*Hình 41 : Phía trước mô hình cánh cửa*

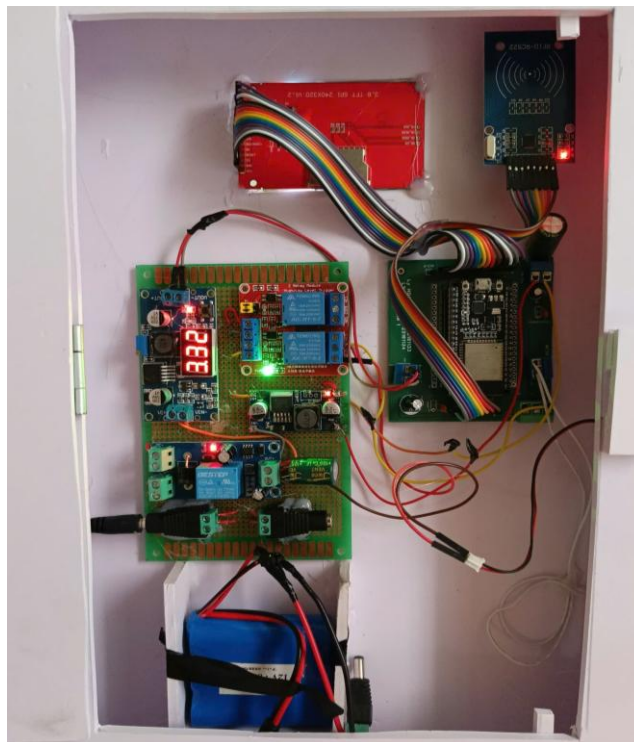


*Phía sau cửa:*



*Hình 42 : Phía sau mô hình cánh cửa*

*Nơi đặt board điều khiển:*



*Hình 43 : Nơi đặt mạch điều khiển và khối nguồn*

#### 4.4. Kết quả kiểm tra các chức năng

Bảng 5 : Kết quả kiểm thử chức năng của hệ thống

| Chức Năng                     | Mô Tả Kiểm Tra                         | Kết Quả     |
|-------------------------------|----------------------------------------|-------------|
| Mở khóa bằng mật khẩu đúng    | Nhập đúng mật khẩu → relay bật         | Đạt yêu cầu |
| Mở khóa bằng mật khẩu sai     | Nhập sai → hiện "SAI MẬT KHẨU"         | Đạt yêu cầu |
| Mở khóa bằng thẻ RFID hợp lệ  | Quét đúng thẻ → relay bật              | Đạt yêu cầu |
| Quét thẻ RFID không hợp lệ    | Quét thẻ lạ → hiện "THẺ KHÔNG HỢP LỆ"  | Đạt yêu cầu |
| Mở khóa qua Firebase App      | Set RELAY_KEY=1 → relay bật trong 2s   | Đạt yêu cầu |
| Tự động khóa khi cửa đóng     | MC-38 phát hiện đóng → relay tắt ngay  | Đạt yêu cầu |
| Timeout 10 giây               | Không đóng cửa → tự tắt relay sau 10s  | Đạt yêu cầu |
| Reset Firebase sau khi đóng   | RELAY_KEY tự về 0 sau khi cửa đóng     | Đạt         |
| Đổi mật khẩu 3 bước           | Nhập cũ → mới → xác nhận → lưu EEPROM  | Đạt yêu cầu |
| Mật khẩu lưu sau khi mất điện | Tắt/bật lại → mật khẩu vẫn giữ nguyên  | Đạt yêu cầu |
| Màn hình TFT cảm ứng          | Chạm các nút → điều hướng đúng trang   | Đạt yêu cầu |
| Hiển thị trạng thái WiFi      | Footer luôn cập nhật WiFi OK / No WiFi | Đạt yêu cầu |

## 4.5. Nhận xét và Hạn chế

### *Nhận xét:*

Về độ chính xác và ổn định khi mở khóa

Cả ba phương thức mở khóa mật khẩu, thẻ RFID và điều khiển từ xa qua ứng dụng đều hoạt động đúng như thiết kế trong quá trình kiểm tra thực tế. Với mật khẩu, hệ thống so sánh chính xác từng ký tự và luôn phản hồi rõ ràng bằng thông báo trên màn hình dù đúng hay sai. Với thẻ RFID, việc quét thẻ diễn ra liên tục ở nền (mỗi 300ms) nên người dùng có thể quét thẻ ngay cả khi đang ở bất kỳ màn hình nào, không cần phải vào đúng một trang riêng mới quét được đây là một điểm thiết kế thuận tiện. Với điều khiển qua app, độ trễ mở khóa thực tế rơi vào khoảng 2–4 giây, phù hợp với chu kỳ polling 2 giây/lần mà tác vụ Firebase thực hiện.

Đặc biệt, cơ chế phát hiện thẻ RFID còn có khả năng tự phục hồi: nếu module RC522 không đọc được thẻ liên tục trong khoảng 10 lần quét (~3 giây), hệ thống sẽ tự động khởi động lại module mà không cần người dùng phải tắt/mở nguồn thủ công điều này giúp hệ thống ít bị "đơ" khi hoạt động lâu dài.

Về logic tự động khóa cửa

Cơ chế theo dõi cảm biến từ MC-38 hoạt động rất tức thời và đáng tin cậy nhờ hai lớp bảo vệ: (1) đọc cảm biến hai lần cách nhau 10ms để lọc nhiễu tín hiệu cơ khí, và (2) chỉ coi là "cửa vừa đóng" khi đã ghi nhận cửa thật sự mở ra trước đó tránh trường hợp khóa nhầm ngay lúc vừa mở relay. Song song đó, hệ thống còn có lớp bảo vệ dự phòng bằng timeout 10 giây, đảm bảo cửa không bao giờ bị bỏ mở quá lâu kể cả khi cảm biến gặp sự cố hoặc người dùng quên đóng cửa. Sự kết hợp hai cơ chế này (đóng cửa tức thời + timeout dự phòng) khiến việc tự động khóa gần như không cần can thiệp thủ công trong mọi tình huống thực tế.

Về kiến trúc phần mềm (FreeRTOS, EEPROM, giao diện)

Việc tách tác vụ gọi Firebase ra chạy riêng trên lõi Core 0 bằng `xTaskCreatePinnedToCore`, trong khi vòng lặp chính xử lý màn hình/cảm ứng/RFID chạy trên Core 1, giúp hai luồng công việc không tranh giành CPU của nhau nhờ vậy giao diện cảm ứng vẫn mượt mà dù đang có yêu cầu HTTP đang chờ phản hồi từ máy chủ. Đồng thời, các cơ chế nhường CPU

đúng cách (vTaskDelay thay vì delay chặn cứng trong task nền) và các khoảng nghỉ nhỏ (30ms) sau khi đóng relay giúp module RFID kịp "nhả" bus SPI trước khi màn hình TFT vẽ lại, tránh được tình trạng xung đột bus SPI hay xé hình vốn dễ gặp khi nhiều thiết bị SPI dùng chung đường truyền.

Mật khẩu và danh sách thẻ RFID đều được lưu vào EEPROM, có kèm cơ chế kiểm tra dữ liệu hợp lệ trước khi sử dụng nếu dữ liệu lỗi hoặc chưa từng ghi (thiết bị mới), hệ thống tự khôi phục về giá trị mặc định an toàn thay vì treo hoặc từ chối hoạt động. Nhờ vậy, dữ liệu cấu hình không bị mất khi mất điện đột ngột, và hệ thống vẫn "sống sót" được ngay cả khi bộ nhớ gặp sự cố nhỏ.

Về giao diện, màn hình TFT cảm ứng với các trang chức năng rõ ràng (trang chủ, nhập mật khẩu, App Firebase, tùy chỉnh, quản lý thẻ) cùng bàn phím số trực quan giúp thao tác dễ dàng, không cần hướng dẫn phức tạp.

#### *Hạn chế:*

Giới hạn về quản lý người dùng:

Hệ thống chỉ lưu được một mật khẩu chung duy nhất cho toàn bộ người dùng (biến savedPassword), chưa hỗ trợ tạo nhiều tài khoản với mật khẩu riêng cho từng người - nghĩa là không thể phân biệt "ai" vừa mở cửa bằng mật khẩu, chỉ biết là có người mở đúng mật khẩu chung. Về thẻ RFID, tuy hệ thống đã hỗ trợ lưu tối đa 5 thẻ và có sẵn giao diện thêm/xóa thẻ ngay trên màn hình (không cần sửa code hay nạp lại firmware), nhưng con số 5 thẻ vẫn là khá ít nếu áp dụng cho một hộ gia đình đông người hoặc văn phòng nhỏ, và giới hạn này đang được cấu hình cứng (MAX\_CARDS = 5) chứ chưa linh hoạt điều chỉnh theo dung lượng EEPROM còn trống.

#### *Chưa có lịch sử ra vào và thông báo realtime:*

Hệ thống hiện không lưu lại bất kỳ nhật ký nào về việc ai đã mở cửa, mở bằng cách nào (mật khẩu/thẻ/app), và vào thời điểm nào mọi sự kiện mở khóa chỉ được in ra Serial (chỉ xem được khi cắm dây USB vào máy tính) rồi mất đi ngay khi tắt cổng debug, không có nơi lưu trữ lâu dài. Điều này khiến việc truy vết khi có sự cố (ví dụ nghi ngờ có người lạ mở cửa) gần như không thể thực hiện được.

Bên cạnh đó, hệ thống cũng chưa có cơ chế gửi thông báo chủ động tới người dùng khi có sự kiện mở cửa xảy ra (ví dụ qua thông báo đẩy trên điện thoại, tin nhắn, hoặc email) người quản lý chỉ biết được trạng thái cửa khi tự mở app lên xem, hoàn toàn bị động.

Bảo mật của kênh điều khiển từ xa còn yếu

Đây là hạn chế nghiêm trọng nhất nếu triển khai vào thực tế: đường dẫn Firebase Realtime Database (RELAY\_KEY.json) hiện không có bất kỳ lớp xác thực hay phân quyền nào bất kỳ ai có được URL này (có thể vô tình lộ ra qua việc chia sẻ code, log, hay bắt được gói tin) đều có thể tự gửi yêu cầu HTTP để đặt giá trị RELAY\_KEY = 1 và mở khóa cửa từ xa mà không cần thông qua ứng dụng chính thức, không cần đăng nhập, và không để lại dấu vết nào để truy vết. Việc thiết bị chỉ mở cửa thật khi người dùng đang đứng ở đúng trang APP trên màn hình (currentPage == PAGE\_APP) có phần nào giảm thiểu rủi ro, nhưng đây vẫn chỉ là một lớp phòng vệ mỏng manh, không thay thế được cho cơ chế xác thực (authentication) và mã hóa đường truyền đúng nghĩa.

*Phụ thuộc hoàn toàn vào kết nối WiFi cho tính năng từ xa:*

Toàn bộ luồng điều khiển qua app dựa vào việc thiết bị kết nối được WiFi và liên lạc được với máy chủ Firebase. Khi mất mạng dù chỉ tạm thời biến wifiConnected chuyển về false và tác vụ Firebase bỏ qua toàn bộ vòng gọi HTTP cho tới khi có mạng trở lại, khiến chức năng mở cửa từ xa bị tê liệt hoàn toàn trong suốt thời gian đó. Mật khẩu và thẻ RFID vẫn hoạt động bình thường vì không phụ thuộc mạng, nhưng đây vẫn là một điểm yếu cần cân nhắc nếu muốn triển khai ở nơi có chất lượng mạng không ổn định, hoặc muốn đảm bảo tính năng từ xa luôn sẵn sàng.

## **CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN**

### **5.1. Kết luận**

Qua quá trình nghiên cứu, thiết kế và thi công, đề tài đã xây dựng thành công một hệ thống khóa cửa thông minh hoàn chỉnh sử dụng vi điều khiển ESP32 làm trung tâm xử lý, tích hợp đồng thời ba phương thức mở khóa: bằng mật khẩu, bằng thẻ RFID, và điều khiển từ xa thông qua ứng dụng kết nối với nền tảng Firebase. Đây là một giải pháp mang tính linh hoạt cao, cho phép người dùng lựa chọn phương thức mở cửa phù hợp tùy theo từng tình



huống thực tế - chẳng hạn dùng thẻ khi ra vào thường xuyên, dùng mặt khẩu khi không mang theo thẻ, hoặc mở cửa từ xa khi có khách đến trong lúc chủ nhà vắng mặt. Toàn bộ các chức năng nêu trên đã được kiểm tra thực nghiệm nhiều lần và hoạt động ổn định, đúng theo các yêu cầu kỹ thuật mà đề tài đặt ra ban đầu. Điểm nổi bật của hệ thống nằm ở sự kết hợp chặt chẽ, đồng bộ giữa phần cứng và phần mềm. Cảm biến từ MC-38 đóng vai trò như một lớp giám sát vật lý độc lập, cho phép hệ thống tự động khóa cửa ngay khi phát hiện cửa đã đóng lại mà hoàn toàn không cần đến bất kỳ thao tác thủ công nào từ người dùng - đây là yếu tố quan trọng giúp hạn chế rủi ro do con người quên khóa cửa. Bên cạnh đó, việc ứng dụng hệ điều hành thời gian thực FreeRTOS để quản lý đa tác vụ song song trên hai lõi xử lý của ESP32 đã được triển khai một cách hiệu quả, giúp tách biệt rõ ràng giữa tác vụ giao tiếp mạng và tác vụ xử lý giao diện, qua đó đảm bảo hệ thống vận hành mượt mà, ổn định và không xảy ra tình trạng xung đột tài nguyên phần cứng dùng chung như bus SPI. Không chỉ dừng lại ở kết quả sản phẩm, quá trình thực hiện đề tài còn mang lại giá trị lớn về mặt học thuật và kỹ năng cho nhóm thực hiện. Thông qua việc trực tiếp thiết kế và gỡ lỗi hệ thống, nhóm đã củng cố và mở rộng đáng kể vốn kiến thức thực tế về lập trình nhúng trên vi điều khiển, nguyên lý hoạt động của giao thức truyền thông SPI, kỹ thuật giao tiếp HTTP với dịch vụ đám mây Firebase, phương pháp quản lý và bảo toàn dữ liệu trên bộ nhớ EEPROM, cũng như kỹ năng thiết kế giao diện người dùng trực quan trên màn hình cảm ứng TFT. Đây chính là nền tảng kiến thức và kinh nghiệm thực chiến quý báu, tạo tiền đề vững chắc để nhóm có thể tự tin tiếp cận và phát triển những hệ thống IoT có quy mô và độ phức tạp cao hơn trong tương lai.

## **5.2. Hướng phát triển**

Mặc dù đã đáp ứng tốt các mục tiêu đề ra, hệ thống hiện tại vẫn còn nhiều dư địa để hoàn thiện trước khi có thể đưa vào ứng dụng thực tế một cách rộng rãi. Dưới đây là những hướng phát triển cụ thể mà nhóm đề xuất cho các giai đoạn tiếp theo.

Về phần bảo mật - đây cần được xác định là ưu tiên hàng đầu và cấp thiết nhất nếu hướng đến việc triển khai hệ thống trong môi trường thực tế, bởi lỗ hổng bảo mật hiện tại của kênh điều khiển từ xa hoàn toàn có thể bị lợi dụng để xâm nhập trái phép. Cần bổ sung ngay cơ chế xác thực người dùng cho Firebase thông qua Firebase Authentication, đồng

thời thiết lập các Security Rules chặt chẽ nhằm giới hạn quyền đọc/ghi dữ liệu chỉ dành cho những tài khoản đã được xác thực hợp lệ, qua đó ngăn chặn triệt để mọi truy cập trái phép từ bên ngoài vào cơ sở dữ liệu điều khiển cửa.

Về tính năng dành cho người dùng, hệ thống cần được nâng cấp cấu trúc dữ liệu để hỗ trợ lưu trữ đồng thời nhiều mật khẩu và nhiều mã UID thẻ RFID, gắn liền với thông tin định danh của từng cá nhân cụ thể - thay vì chỉ dùng chung một mật khẩu duy nhất như hiện tại - nhằm phục vụ tốt hơn cho các hộ gia đình đông thành viên hoặc môi trường văn phòng có nhiều người ra vào. Song song đó, nên bổ sung chức năng ghi nhận và đồng bộ lịch sử ra vào theo thời gian thực lên Firebase (bao gồm thông tin người mở, phương thức mở, và mốc thời gian), giúp chủ nhà có thể chủ động theo dõi, tra cứu và kiểm soát hoạt động ra vào một cách minh bạch, thuận tiện hơn rất nhiều so với việc chỉ đọc log qua cổng Serial như hiện tại.

Về thông báo và giám sát từ xa, nhóm đề xuất tích hợp thêm Telegram Bot hoặc Firebase Cloud Messaging nhằm gửi cảnh báo tức thời đến điện thoại người dùng ngay khi có sự kiện mở cửa xảy ra, hoặc khi hệ thống phát hiện các thao tác bất thường có dấu hiệu xâm nhập trái phép - chuyển hệ thống từ mô hình giám sát bị động (chỉ biết khi tự kiểm tra) sang mô hình cảnh báo chủ động theo thời gian thực.

Về giao diện điều khiển, thay vì phải thao tác trực tiếp và thủ công trên Firebase console như hiện nay - vốn không thân thiện với người dùng phổ thông - nhóm đề xuất phát triển một ứng dụng di động chuyên biệt, có thể xây dựng trên nền tảng Flutter hoặc MIT App Inventor. Ứng dụng cần được thiết kế để hiển thị trạng thái cửa theo thời gian thực một cách trực quan, đồng thời cho phép người dùng thực hiện thao tác mở khóa chỉ bằng một lần chạm duy nhất, nhằm tối ưu hóa trải nghiệm sử dụng hàng ngày.

Về phần cứng, module ESP32-CAM vốn đã được tích hợp sẵn trên mạch nhưng chưa được khai thác cần được đưa vào sử dụng để bổ sung tính năng nhận diện khuôn mặt thông qua thư viện ESP-WHO, qua đó nâng cao thêm một tầng bảo mật sinh trắc học mới cho hệ thống, kết hợp cùng ba phương thức xác thực hiện có. Bên cạnh đó, cần tiếp tục kiểm tra, đo đạc và hoàn thiện mạch nguồn dự phòng sử dụng pin 18650, nhằm đảm bảo hệ thống

vẫn có thể duy trì hoạt động liên tục và không bị gián đoạn ngay cả trong tình huống mất điện lưới đột ngột – một yếu tố quan trọng đối với thiết bị an ninh vận hành 24/7.

Về trải nghiệm người dùng, cần nâng cấp bộ font chữ hiển thị trên màn hình TFT để hỗ trợ đầy đủ tiếng Việt có dấu, giúp giao diện trở nên thân thiện, dễ đọc và chuyên nghiệp hơn thay vì chỉ hiển thị được tiếng Việt không dấu như hiện tại. Ngoài ra, việc bổ sung tính năng cập nhật firmware từ xa qua WiFi (OTA – Over-The-Air Update) cũng là một hướng phát triển thiết yếu, giúp công tác bảo trì, vá lỗi và nâng cấp tính năng cho hệ thống trong tương lai trở nên nhanh chóng và thuận tiện hơn rất nhiều, mà không đòi hỏi phải tháo thiết bị ra để kết nối trực tiếp qua cáp USB mỗi khi cần cập nhật.

## TÀI LIỆU KHAM KHẢO

- [1] R. D. R. Bueno, V. A. da Cunha, and V. N. Frazão, *Fechadura Eletrônica Inteligente com ESP32*, Trabalho de Conclusão de Curso (Técnico em Automação Industrial), Escola Técnica Estadual Dep. Ary de Camargo Pedroso, Centro Paula Souza, Piracicaba, São Paulo, Brazil, Jun. 2025
- [2] A. D. Putra, “Implementasi Door Lock Menggunakan RFID, Bluetooth, dan Wi-Fi pada Pintu Berbasis Android,” Tugas Akhir, Program Studi Elektronika Industri, Jurusan Teknik Elektro, Politeknik Negeri Jakarta, Depok, Indonesia, 2023.
- [3] P. V. Vuong, P. D. Minh, and L. H. Toan, “Khóa cửa thông minh sử dụng mật khẩu và vân tay,” Đồ án Cơ sở ngành, Khoa Điện – Điện tử, Trường Đại học Công nghiệp Hà Nội, Hà Nội, Việt Nam, 2024.
- [4] H. T. Thanh, H. T. Thanh, N. T. T. Linh, and K. N. Ngoc, “Hệ thống khóa cửa thông minh,” Báo cáo Đồ án Hệ thống nhúng, [Học Viện Công Nghệ Bưu Chính Viễn Thông], Việt Nam, 2025.
- [5] H. P. Thuan, N. H. Thong, T. T. Thanh, D. N. Sy, and D. V. T. Dat, “Thiết kế khóa cửa thông minh nhận diện khuôn mặt,” Báo cáo Đồ án, [Đại Học Vinh Viện Kỹ Thuật và Công Nghệ], Việt Nam, 2026.
- [6] Espressif Systems, "ESP32 Series Datasheet," Version 3.0, 2019.

